MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA
FACULTY OF AUTOMATION AND COMPUTER SCIENCE

A STUDY OF HOMOMORPHIC ENCRYPTION APPROACHES FOR PRIVACY-PRESERVING LOCATION-BASED SERVICES

LICENSE THESIS

Graduate: **Mariana-Ionela MUNTIAN**

Supervisor: **Conf. Dr. Eng. Adrian Viorel COLEŞA**

2026

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Graduate: **Mariana-Ionela MUNTIAN**

A STUDY OF HOMOMORPHIC ENCRYPTION APPROACHES FOR PRIVACY-PRESERVING LOCATION-BASED SERVICES

1. **Project proposal:** *This thesis aims to compare the performance and limitations of multiple homomorphic encryption schemes for location-based services and to establish the contexts in which HE becomes practically usable.*

2. **Project contents:** *Introduction, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, Detailed Design and Implementation, Testing and Validation, Conclusions, Bibliography, Appendices*

3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department

4. **Consultants:** Prof. Dr. Eng. Adrian Viorel COLEŞA, PhD. István-Attila CSÁSZÁR

5. **Data of issue of the proposal:** October 1, 2025

6. **Data of delivery :** July 10, 2026

Graduate: Mariana-Ionela Muntian

Supervisor: Adrian Viorel Coleşa

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Declarație pe proprie răspundere privind autenticitatea lucrării de licență

Subsemnata **Mariana-Ionela Muntian**, legitimată cu cartea de identitate seria **KS**, **nr. 725539** (conform copiei anexate prezentei declarații, copie semnată şi certificată "conform cu originalul"), autorul lucrării **Studiu privind abordările de criptare homomorfă pentru servicii de localizare cu protejarea confidențialității**, elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Programul de studiu **Calculatoare cu predare in lb. engleză** din cadrul Universității Tehnice din Cluj-Napoca, sesiunea iulie a anului universitar 2026, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele şi pe baza informațiilor obținute din surse care au fost citate, în textul lucrării şi în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române şi a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*. Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar şi se va dori verificarea autenticității lucrării mele să fiu identificat şi verificat în baza datelor declarate de mine şi conform copiei documentului de identitate menționat mai sus.

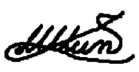
Data

10/07/2026

Nume, Prenume

Muntian, Mariana-Ionela

Semnătura



Contents

Chapter 1 Introduction - Project Context **1**
1.1 Project Context . . . 1
1.1.1 Privacy Risks in Location-Based Services . . . 1
1.1.2 Homomorphic Encryption . . . 1
1.1.3 Homomorphic Encryption for Location-Based Services . . . 1
1.2 Motivation . . . 2
1.3 Proposed Method . . . 2
1.3.1 Finding the Nearest Point of Interest . . . 2
1.3.2 Checking Employee Location Compliance . . . 3
1.3.3 Detection of Account Takeover . . . 3
1.4 Report's Structure . . . 4

Chapter 2 Project Objectives **5**
2.1 Finding the Nearest Point of Interest . . . 5
2.2 Checking Employee Location Compliance . . . 6
2.3 Detection of Account Takeover . . . 6

Chapter 3 Bibliographic Research **7**
3.1 Related Work . . . 7
3.1.1 Private Nearest Point-of-Interest Search and Location Sharing . . . 7
3.1.2 Detection of Account Takeover on RBA dataset . . . 9

Chapter 4 Analysis and Theoretical Foundation **11**
4.1 Foundational Concepts in Homomorphic Encryption . . . 11
4.1.1 Definition and Properties . . . 11
4.1.2 Terminology . . . 11
4.1.3 Classification of Homomorphic Encryption . . . 12
4.2 CKKS: Approximate Fully Homomorphic Encryption . . . 13
4.2.1 Scheme Overview . . . 13
4.2.2 Computational Properties Used . . . 13
4.2.3 Applicability to Location Data . . . 14
4.3 FHEW: Fastest Homomorphic Encryption in the West . . . 16
4.3.1 Scheme Overview . . . 16
4.3.2 Computational Properties Used . . . 16
4.4 Scheme Switching: From CKKS to FHEW . . . 17
4.5 TFHE: Fast Fully Homomorphic Encryption over the Torus . . . 18
4.5.1 Scheme Overview . . . 18
4.5.2 Computational Properties Used . . . 18
4.6 Location Compliance Verification: Theoretical Protocol . . . 19
4.6.1 Parties and Trust Model . . . 19
4.6.2 Key Setup . . . 19
4.6.3 Encryption Phase . . . 20
4.6.4 Server-Side Computation . . . 20
4.6.5 Threshold Decryption . . . 20
4.6.6 Compliance Evaluation . . . 21

4.7 Detection of Account Takeover . . . 21
4.7.1 eXtreme Gradient Boosting (XGBoost) . . . 21
4.7.2 Evaluation Metrics for Imbalanced Classification . . . 22
4.7.3 Privacy Model . . . 23
4.8 Communication Protocol between Client and Server . . . 24

Chapter 5 Detailed Design and Implementation 25
5.1 System Design and Architecture . . . 25
5.1.1 Conceptual Architecure . . . 25
5.1.2 Deployment Diagram . . . 25
5.2 Preprocessing of OpenStreetMap (OSM) Data . . . 26
5.3 Finding the Nearest Point of Interest: CKKS + FHEW . . . 29
5.3.1 Parameters and Key Setup . . . 29
5.3.2 Server-Side Computation . . . 30
5.3.3 Scheme Switching . . . 31
5.3.4 Client-Side Decryption . . . 31
5.3.5 Pipeline . . . 31
5.4 Finding the Nearest Point of Interest: TFHE . . . 31
5.4.1 Coordinate Preprocessing . . . 31
5.4.2 Circuit Design . . . 32
5.4.3 Key Compression . . . 34
5.4.4 Client-Server Protocol . . . 34
5.4.5 Pipeline . . . 36
5.5 Location Compliance Verification . . . 36
5.5.1 Parameters and Key Setup . . . 36
5.5.2 Company Server Computations . . . 37
5.5.3 Employees server computations . . . 38
5.5.4 Encrypted Compliance Check . . . 39
5.5.5 Threshold Decryption and Fusion . . . 41
5.5.6 Decision Logic . . . 42
5.6 Detection of Account Takeover . . . 43
5.6.1 Pipeline . . . 43
5.6.2 Dataset Analysis & Memory Usage Optimization . . . 44
5.6.3 Feature Engineering . . . 48
5.6.4 Train-Test Split . . . 50
5.6.5 Handling Class Imbalance . . . 51
5.6.6 Model Training . . . 51
5.6.7 Inference . . . 54
5.7 Transmission Protocol . . . 55
5.7.1 Interface Definition . . . 55
5.7.2 Service Provider support for gRPC . . . 56
5.7.3 Client support for gRPC . . . 56

Chapter 6 Testing and Validation 57
6.1 Finding the Nearest Point of Interest . . . 57
6.2 Location Compliance Verification: Theoretical Protocol . . . 59
6.3 Detection of Account Takeover . . . 60

Chapter 7 User's Manual **65**
7.1 Development Environment 65
7.2 Libraries and Dependencies 65
7.3 Execution Configuration 66
7.4 Starting the Application 66

Chapter 8 Conclusions **69**
8.1 Contributions 69
8.2 Analysis of the Results 69
8.2.1 Finding the Nearest Point of Interest 69
8.2.2 Location Compliance Verification: Theoretical Protocol 70
8.2.3 Detection of Account Takeover 70
8.3 Future Development 70

Bibliography **71**

Appendix A Relevant Code sections **72**

Appendix B Other Relevant Info **73**

Chapter 1. Introduction - Project Context

1.1. Project Context

1.1.1. Privacy Risks in Location-Based Services

Nowadays, many applications and critical infrastructure systems rely on location-based services. Examples include GPS-based navigation applications (e.g., Google Maps), workplace platforms that may use location for compliance verification (e.g., Microsoft Teams), authentication systems that detect suspicious logins based on geographic inconsistencies and device fingerprints (e.g., Okta, Microsoft, banking authentication systems), and logistics platforms (e.g., courier services, ride-sharing).

Although these services are valuable, using them comes at the price of exposing to the service provider, user sensitive information such as location and device fingerprints. One notable example is that in 2022, Kochava, a U.S. based mobile data and advertising analytics company, was sued by the U.S. Federal Trade Commission for selling precise geolocation data about millions of users. Similarly, Microsoft was accused of continuing to monitor users' location, even after they had disabled location tracking. Additionally, in 2024, another tech company agreed to delete billions of browsing records after being accused of tracking web activity, even in Incognito mode. A similar privacy case occurred in 2025, when the company agreed to pay $1.375 billion to the state of Texas in order to solve claims involving geolocation, biometric data, and Incognito searches.

1.1.2. Homomorphic Encryption

Homomorphic Encryption is a type of encryption that allows computations to be performed on encrypted data. After decrypting the result of the computation, the output should be identical or very similar to the result obtained by performing the same computation on plaintext data.

While these properties of Homomorphic Encryption make HE seem ideal for solving many privacy concerns, it also has some substantial limitations. The first limitation is that only certain types of operations can be performed on encrypted data, depending on the type of HE and on the specific scheme. Most importantly, both the time and memory requirements of HE are extremely high.

1.1.3. Homomorphic Encryption for Location-Based Services

An initial idea would be that Homomorphic Encryption can be used to protect the user's precise location in location-based services by simply transforming all operations of classical location-based services into HE operations.

However, let us consider applications such as Google Maps, which use complex routing algorithms that must be as efficient as possible and respond quickly to user

input. Such applications also need to recenter while the user is moving at high speed, for example while driving. Due to the high computational cost and latency of HE, it would be impractical to implement such applications fully using HE. As a consequence, location-based services that rely on HE must be specially designed to support it, while remaining practical and relevant for the user.

1.2. Motivation

The primary motivation of this thesis is to identify and define location-based service scenarios and systems that can be practically usable while preserving the privacy of the user's location through Homomorphic Encryption. A secondary motivation is to analyze and compare the performance of multiple encryption schemes in terms of execution time, memory usage, limitations, and accuracy for each service. Lastly, the thesis aims to integrate the proposed services with real-world datasets by applying the necessary preprocessing techniques to transform the raw data into a suitable form for HE computation.

The results of this research will contribute to a better understanding of the benefits and limitations that HE can bring to location-based services. Moreover, they will help determine whether HE is realistically suitable for specific scenarios or whether other privacy-preserving techniques should be considered.

1.3. Proposed Method

This section briefly presents the proposed system and the methods used. The system is composed of one frontend application for the client, two backend services, namely the client server and the service provider server, and the corresponding databases. From a logical perspective, the system is divided into three main modules, which are as follows:

1.3.1. Finding the Nearest Point of Interest

This module allows the user to query the nearest point of interest from a selected category (e.g. hospital, school, stationery, post office, cosmetics, or gallery) using a selected homomorphic encryption library (OpenTFHE or Concrete) and is composed of the following components:

- **Client-side query processing:** it receives the user's plaintext data and generates the private, public, and evaluation keys, encrypts the coordinates with the public key, and sends the encrypted query, category, public key, and evaluation key to the service provider through gRPC. After receiving the encrypted result, the client server decrypts it using the private key and sends the final result back to the frontend.

- **Service provider processing:** The service provider has access to a preprocessed dataset containing all points of interest. It filters the records based on the selected category, computes the encrypted distance between each candidate location and the encrypted user location, and identifies the minimum distance. The service provider returns the encrypted result containing the closest place from the selected category.

- **Dataset preprocessing:** The OpenStreetMap dataset was reduced to locations from Cluj-Napoca. From the original data, only the fields relevant to this thesis were extracted into a single table. The coordinates, initially represented in degrees, were transformed into a coordinate space that supports distance computation and reduced to smaller numerical values suitable for HE.

- **Simulator:** To compare the performance of the Concrete and OpenTFHE libraries, a simulation is performed using 10 randomly selected points as user coordinates for each target category. The simulator measures latency and memory overhead for key generation, encryption, decryption, and encrypted computation. It also computes deviation of each scheme from the true result.

1.3.2. Checking Employee Location Compliance

The second module checks whether an employee is located in the declared country, at the declared home address, or at the office. Each company can define its own policy, including the office address and the accepted radius, and run a compliance check for each of its employees.

During this procedure, both the employee's location and the company's reference location remain private by using Threshold CKKS, more specifically OpenFHE's multiparty threshold variant of CKKS. Both parties participate in key generation, and both are required to participate in decrypting the result received from the service provider.

To check the correctness of the results, a check is run on plaintext in parallel with the homomorphic encryption pipeline. Multiple scenarios are tested to ensure that, even in edge cases, such as when the employee sits in the corner of the office, the algorithm performs accurately.

1.3.3. Detection of Account Takeover

The proposed method uses a Machine Learning approach to detect suspicious login attempts that may indicate account takeover. The selected dataset is the **Login Data Set for Risk-Based Authentication** (RBA), which was collected from a single sign-on online service from Norway.

The model was trained using XGBClassifier in the classical plaintext setting. In addition, a Concrete-ML-compatible variant was prepared to support encrypted inference. During testing, inference is performed on plaintext data, on HE simulated, and on HE executed. HE simulated means that the inference is performed on the circuit from Concrete-ML, but the data is in plaintext, while for HE executed, the data is encrypted. This comparison is used to analyze whether encrypted inference has the same performance as the plaintext model and also to compute the gap that quantization creates.

Therefore, this module has a dual purpose: to develop a performant model for risk-based authentication and also to analyze the performance gap that homomorphic encryption adds. The main metric that the models try to improve is recall, as it is more important to block real attacks than to keep false positive alarms low.

1.4. Report's Structure

The paper is structured around eight chapters, which discuss in detail the main aspects of this thesis.

- **Chapter 1 - Introduction:** discusses the current privacy concerns that lay the foundation for motivating the research and development of the chosen topic. It also presents the methods proposed to address these privacy concerns.

- **Chapter 2 - Project Objectives:** presents the main research objectives of the thesis in the context of privacy-preserving location-based services.

- **Chapter 3 - Bibliographic Research:** describes current scientific studies on Homomorphic Encryption in a broader context and explains how they helped in identifying possible solutions to the discussed problem.

- **Chapter 4 - Analysis and Theoretical Foundation:** introduces the main concepts of Homomorphic Encryption and explains the encryption schemes used in this thesis, such as TFHE, CKKS, FHEW, and multiparty threshold schemes, together with other relevant theoretical elements.

- **Chapter 5 - Detailed Design and Implementation:** describes the proposed architecture, the design of each module, and the technical implementation details.

- **Chapter 6 - Testing and Validation:** presents the comparative results of the Concrete and OpenFHE libraries, the accuracy of the compliance-checking result, and the results of the implemented model for detecting account takeover, while also acknowledging the limitations of each solution.

- **Chapter 7 - User's Manual:** describes the tools, libraries, and hardware support needed to install and run the application.

- **Chapter 8 - Conclusions:** presents the contributions of this thesis, as well as its limitations and the remaining research gaps.

Chapter 2. Project Objectives

The questions around which the direction and development of this thesis were structured are as follows:

- In which location-based service scenarios can Homomorphic Encryption be used in a practical and meaningful way?
- Which Homomorphic Encryption schemes are most suitable for computations over location data, and how do their configuration parameters influence performance and accuracy?
- How close are the results obtained through encrypted computation to the results obtained through plaintext computation?
- What are the main challenges and practical limitations of using Homomorphic Encryption for privacy-preserving location-based services?

The first question led to the division of the system into three independent modules, each exploring a different context in which Homomorphic Encryption could be applied:

2.1. Finding the Nearest Point of Interest

The main objectives of this module are:

1. To rigorously preprocess the data extracted from OpenStreetMap, so that it is transformed into a suitable format for efficient use in Homomorphic Encryption-based computations.

2. To research existing Homomorphic Encryption schemes suitable for supporting distance computation and nearest-neighbor selection, and to adapt their corresponding libraries to the available hardware resources and operating system constraints. The parameter configuration of each scheme must be determined with the aim of preserving an appropriate balance between computational efficiency and result accuracy.

3. To construct an experimental simulation framework that enables the selected schemes to be compared in terms of execution time, memory usage, and correctness of the results, measured through the average deviation from the plaintext distance computation.

2.2. Checking Employee Location Compliance

Similar objectives were defined for the second scenario, with the distinction that this scenario involves two independent parties:

1. To investigate suitable multiparty computation and privacy-preserving cryptographic schemes, and to design and implement a workflow that enables location compliance verification between an employer and an employee. The workflow aims to preserve the privacy of both parties by checking whether the employee satisfies the required location condition without revealing the exact location of either party to each other or to the server.

2. To evaluate the correctness of the proposed encrypted computation by comparing the compliance flags obtained in the privacy-preserving workflow with the corresponding flags produced through plaintext computation.

2.3. Detection of Account Takeover

Finally, for the scenario focused on detecting account takeover through login activity analysis, the following objectives were defined:

1. To develop a machine learning model capable of achieving competitive performance on the "Login Data Set for Risk-Based Authentication (RBA)", in comparison with the results reported in existing research on risk-based authentication.

2. To perform inference both in the classical plaintext setting and on homomorphically encrypted data, in order to compare the obtained results in terms of recall, precision, execution time, and memory usage.

3. To identify and apply strategies for reducing the memory usage of the development notebook without negatively affecting the predictive performance of the model. This objective is particularly relevant because the implementation must be executed inside a Docker environment, which is required to ensure compatibility between Windows and the Concrete-ML library.

Chapter 3. Bibliographic Research

3.1. Related Work

3.1.1. Private Nearest Point-of-Interest Search and Location Sharing

Several previous works were done on trying to preserve the user's location from LBS (Location-Based Services), but there are not many that address this problem by using homomorphic encryption. However, the paper that inspired this module of the thesis is *Universal Location Referencing and Homomorphic Evaluation of Geospatial Query* [1].

This targets the same threat model as the thesis, namely, the service provider has access to the plaintext database containing the points of interest, and the position of the client remains private by encrypting it.

The authors proposed a framework that allows the evaluation of location queries directly on encrypted data. In the system they proposed, the coordinates are first encoded using space-filling curves, which is a dimensionality reduction technique that preserves locality and maps two-dimensional coordinates to a one-dimensional value. The scheme that was chosen was from the family of somewhat homomorphic encryption (SHE).

There are three types of algorithms proposed, corresponding to different types of queries:

1. **"Where is Alice?"**: A friend of Alice is allowed to see Alice's location if she has granted him access. She can set the level of detail that he is allowed to see.

2. **"Who is nearby?"**: Alice can request to see which friends are in the nearby area.

3. **"How close are Alice and Bob?"**: The server tries to find the smallest bounding box that contains both users.

Query Type 1: Private Retrieval of a Friend's Location

The procedure for the first query is the cheapest and is as follows:

- Alice encrypts her location under a joint public key and defines a bit mask for each friend, which controls how precisely the friend can see her location. The longer the prefix of the mask, the higher the access of the friend will be.
- Alice sends the encrypted location and bit mask to the service provider.
- When Bob asks, "Where is Alice?", the service provider performs a multiplication between the mask Alice provided for Bob and her encrypted location.
- The server returns the result of the multiplication to Bob. Therefore, this is the only multiplication in this procedure, which means that it requires a multiplicative depth of one for homomorphic encryption.

- Bob decrypts the result via a threshold protocol and needs a decryption server that has access to all users' secret shares so that it can send Bob the decrypted result.

The second module, Location Compliance Verification, was built upon this idea. However, the difference is that, in my implementation, there is no bit mask, as the precise encrypted locations must arrive at the server, while only the result of the compliance check is shared with the users. Similar to the paper by Asma Aloufi, multiparty threshold decryption is used. However, instead of using a decryption server, the result is sent to each party for partial decryption, after which the partial results are combined.

Query Type 2: Private Identification of All Nearby Friends

The second type of query addressed in the paper is somehow built on the first type. The first part of the procedure is identical: Alice shares her encrypted location and encrypted bit mask for each friend. However, the difference is that Bob asks, "Which friends are nearby?"

When Bob sends the request to the server, he must also define a region bit-mask, which is different from the privacy bit mask discussed previously. This mask controls how large the nearby area should be. For example, if the prefix of the mask is short, the area will be large. When the request containing the mask reaches the server, the server first performs, as previously described, the multiplication between each friend's encrypted location and the privacy bit mask defined by that friend for Bob. The result of this multiplication is the location of the friend that Bob is allowed to see. This result is then multiplied again by the region mask sent by Bob. Let us call this result Enc_j and suppose that the region bit mask has its first k bits set to 1.

To evaluate whether a friend is inside the region, a subtraction must be performed between Bob's encrypted location and Enc_j. If user j is in the same area as Bob, then the first k bits of the subtraction result must be zero. Bob will receive a list of encrypted results, and after decrypting them, he can see whether each friend is inside or outside the region.

This procedure requires a multiplicative depth of two, and it also needs to check every other user (friend), so it is more expensive than the first query.

Query Type 3: Private Distance Computation Between Two Friends

In summary, the procedure for this query uses a trick: instead of computing the distance between the two users directly, it attempts to build a box (region) that contains both users. The size of the region is determined by comparing the prefixes of the two users' encrypted locations. The longer the matching prefix is, the closer the users are. The diagonal of the box indicates the maximum possible distance between the users. Building the box is a complex procedure that will not be discussed further here, as this query type was not implemented in the thesis.

The algorithm used in the third query notably reduces the multiplicative depth by half. This optimisation reduces the computational overhead.

While the paper provides the same privacy guarantee as the thesis, the methods used are different. Apart from the ones already discussed, the authors preprocessed the coordinates by geohashing them using space-filling curves, while this thesis transformed

them into Web Mercator metres to make the computation of the Euclidean distance possible.

Additionally, the scheme used in *Universal Location Referencing and Homomorphic Evaluation of Geospatial Query* [1] is BGV (Brakerski–Gentry–Vaikuntanathan). BGV is able to perform arithmetic only on integer numbers, and if it were to be used for floating-point numbers, additional preprocessing would have been required. Therefore, the choice for this thesis was to use CKKS instead, which natively supports floating-point arithmetic.

Beyond homomorphic encryption, the literature presents several other approaches. Some techniques include spatial cloaking and k-anonymity. Through these techniques, the user's location is hidden, as he or she reports a region instead of a precise point, in which at least (k) other users exist. However, these workflows require a trusted anonymizer. Additionally, differential privacy can be employed to add calibrated noise to the queries, but this limits precision.

3.1.2. Detection of Account Takeover on RBA dataset

Traditional Risk-Based Authentication and the Original Dataset

The dataset, *Login Data Set for Risk-Based Authentication* [2], by Stephan Wiefling, Paul René Jørgensen, Sigurd Thunem, and Luigi Lo Iacono, originates from real-world logins to an SSO service in Norway, monitored between February 2020 and February 2021. The dataset includes more than 3.3 million users and 33 million login attempts. However, the public dataset was synthesised so that it still reflects real behaviour, but does not allow the re-identification of customers, as they used this SSO service to access sensitive information, such as billing information and cloud storage.

The features of the dataset are the following:

- IP Address
- Country
- Region
- City
- Autonomous System Number (ASN)
- User Agent String
- Operating System Name and Version
- Browser Name and Version
- Device Type
- User ID
- Login Timestamp
- Round-Trip Time (RTT)
- Login Successful
- Is Attack IP
- Is Account Takeover

The original dataset was proposed in 2022 in the paper *Pump Up Password Security! Evaluating and Enhancing Risk-Based Authentication on a Real-World Large-Scale Online Service*[3].

The paper from 2022 attempts to evaluate multiple machine learning classifiers for separating legitimate logins from account takeovers. The evaluated models were isolation forest, one-class SVM, robust covariance and local outlier factor, but the conclusion was that all of them were impractical. The reason was that many of the features are sparse and categorical, which reduces accuracy and increases latency. As a consequence, their approach was to use the Freeman statistical model while choosing the configuration parameters using machine learning.

An important point is that the paper aimed to identify approaches that could block as many attacks as possible, rather than necessarily predicting whether a login

attempt was an attack. Therefore, the configuration parameters provided for different use cases indicate how the SSO service should be configured to reduce the number of successful attacks. For example, blocking 99.5% of attacks required users to be prompted to reauthenticate at every second login.

They authors highlighted that pairing the current record with the previous one from the login history significantly improves detection, as one isolated record cannot provide much information.

A Machine Learning Approach to Risk-Based Authentication

The paper *MLE-RBA: A Machine Learning-Empowered Risk-Based Authentication Algorithm* [4] approaches RBA by proposing a machine learning model and comparing it to the baseline, which is Freeman's statistical model.

The model used was a LightGBM classifier, trained on a user login dataset, and the results indicated that it outperformed Freeman's model. This challenges the conclusion of the study *Pump Up Password Security! Evaluating and Enhancing Risk-Based Authentication on a Real-World Large-Scale Online Service*[3], discussed above, which concluded that all machine learning models performed poorly and that Freeman's model is a better solution.

The dataset used is the same as the one discussed above, but the model's performance was measured using the Equal Error Rate (EER) and other metrics.

Chapter 4. Analysis and Theoretical Foundation

4.1. Foundational Concepts in Homomorphic Encryption

4.1.1. Definition and Properties

Homomorphic Encryption is a type of encryption that allows computations to be performed on encrypted data. Computation over encrypted data is often enabled by using an evaluation key shared with the untrusted party, although not all schemes require such a key. After decrypting the result of the computation, the output should be identical or very similar to the result obtained by performing the same computation on plaintext data.

$$\mathrm{Enc}(a) \cdot \mathrm{Enc}(b) = \mathrm{Enc}(a + b) \tag{4.1}$$

$$\mathrm{Enc}(a)^b = \mathrm{Enc}(a \cdot b) \tag{4.2}$$

The above equations, provided as examples, are properties of the Paillier cryptosystem.

Homomorphic refers to the algebraic homomorphism. The encryption function corresponds to the homomorphism between plaintext and ciphertext spaces, while the decryption function corresponds to the homomorphism between ciphertext and plaintext spaces.

4.1.2. Terminology

The terms defined below are essential for understanding the concepts that will be discussed in the following sections:

Plaintext: the data before encryption, in a readable format.

Ciphertext: the data after encryption, which cannot be interpreted without decryption.

Encryption: the process in which a function is applied to the plaintext in order to transform it into ciphertext.

Decryption: the process in which a function is applied to the ciphertext in order to transform it back into plaintext.

Secret Key: a key that allows the transformation of ciphertext back into plaintext, but which must not be shared.

Public Key: a key that allows the transformation of plaintext into ciphertext and can be shared.

Evaluation Key: a key that is needed in order to perform operations on encrypted data.

Noise: a random value added during encryption in order to enhance security. If noise were not added, then the result of encryption would be deterministic.

Noise Growth: during operations on encrypted data, the noise increases and must be kept under a specific threshold, so that it does not dominate the actual result.

Bootstrapping: a technique used to reduce the accumulated noise, so that the result remains accurate.

Depth: the maximum number of sequential operations that are performed in one computation. Parallel operations are not considered.

Polynomial Degree: defines the mathematical space of encrypted computations. It controls the level of security and the size of the ciphertext. A higher degree means a higher security level, but also slower computation time.

Precision: the number of decimal places that are preserved during encrypted computations.

4.1.3. Classification of Homomorphic Encryption

Homomorphic Encryption schemes can be classified based on the types of operations they support into the following categories:

1. **Partially Homomorphic Encryption:** Schemes in this category can evaluate circuits with only one type of gate/operation (e.g. addition or multiplication or subtraction).

2. **Somewhat Homomorphic Encryption:** While these schemes support the evaluation of two types of gates, the depth of the circuits is limited by the accumulation of noise.

3. **Leveled Fully Homomorphic Encryption:** Allows computation on arbitrary circuits with multiple operation types, but requires the circuit depth to be fixed in advance.

4. **Fully Homomorphic Encryption:** This is the strongest form of Homomorphic Encryption, as it supports multiple types of operations and enables the evaluation of

circuits with unbounded depth. The noise growth that appears during deep computations is controlled through bootstrapping.

In the proposed solution, Leveled Fully Homomorphic Encryption was used through the Cheon-Kim-Kim-Song (CKKS) [5] scheme, while Fully Homomorphic Encryption was used through the Fastest Homomorphic Encryption in the West (FHEW) [6] and Torus Fully Homomorphic Encryption (TFHE) [7] schemes. These schemes will be discussed in more detail in Chapter 4.2, Chapter 4.3, and Chapter 4.5.

4.2. CKKS: Approximate Fully Homomorphic Encryption

4.2.1. Scheme Overview

The CKKS [5] scheme, whose name comes from the researchers who designed it, namely Cheon, Kim, Kim, and Song, is a leveled Homomorphic Encryption scheme for approximate arithmetic. It operates on real and complex numbers, but it treats the least significant bits of the plaintext as negligible, so that the caused error remains controlled and the computation becomes more efficient.

4.2.2. Computational Properties Used

Although CKKS provides multiple properties, for the sake of keeping the discussion straight to the point, only the properties applied in the proposed solution will be discussed.

Addition

For addition, the main constraint is that both ciphertexts must be at the same level.

$$\mathrm{Enc}(a) \oplus \mathrm{Enc}(b) \approx \mathrm{Enc}(a+b) \tag{4.3}$$

$$\mathrm{Enc}(a) \oplus b \approx \mathrm{Enc}(a+b) \tag{4.4}$$

Negation

$$\ominus\mathrm{Enc}(a) \approx \mathrm{Enc}(-a) \tag{4.5}$$

Multiplication

$$\mathrm{Enc}(a) \otimes \mathrm{Enc}(b) \approx \mathrm{Enc}(a \cdot b) \tag{4.6}$$

Multiplication must be followed by relinearization, which reduces the degree of the resulting ciphertext after multiplication back to a lower degree, and by rescaling.

Rescaling

As mentioned in Section 4.2.1, before the plaintext is encrypted, it is multiplied by a scaling factor Δ. Therefore, what gets encrypted is $\text{Enc}(\Delta \cdot a)$. After a multiplication, this factor Δ is squared, which is one of the reasons why the output of a multiplication becomes more noisy:

$$\text{Enc}(\Delta \cdot a) \otimes \text{Enc}(\Delta \cdot b) = \text{Enc}(\Delta^2 \cdot a \cdot b) \tag{4.7}$$

Rescaling is an operation that transforms the squared factor back to its initial form:

$$\text{Enc}(\Delta^2 \cdot a \cdot b) \xrightarrow{\text{rescaling}} \text{Enc}(\Delta \cdot a \cdot b) \tag{4.8}$$

4.2.3. Applicability to Location Data

The dataset OpenStreetMap, after preprocessing looks in the following manner:

id	name	category	lat	lon	lat_m	lon_m	city
2329184457	Anticariat Academic	books	46.77183690020694	23.5934157	13911.2425694242	9407.02179802628	Cluj-Napoca
2329181777	Pell Art	clothes	46.771959500206904	23.5938896	13931.1690969476	9459.77610471332	Cluj-Napoca
2177295925	CEC Bank	atm	46.77282990020673	23.594649199999996	14072.6389946081	9544.33438991942	Cluj-Napoca
1583741378	Raiffeisen Bank	bank	46.773434300206624	23.5926063	14170.8761004163	9316.91980217816	Cluj-Napoca
1704096419	Delissima	bakery	46.774642600206384	23.592631999999995	14367.2723490233	9319.78071309114	Cluj-Napoca
4128113945	Farmacia Balanis	pharmacy	46.75280780021065	23.5789997	10818.9384195711	7802.24001875054	Cluj-Napoca
10803867230	Regina Maria	clinic	46.75410400021038	23.584904299999998	11029.5413952349	8459.5370840882	Cluj-Napoca

Table 4.1: Sample of the processed OpenStreetMap POI dataset

The objective is to compute the distance between the encrypted location of the user and each point of interest that belongs to the category requested by the user. The selection of the minimum distance will be discussed in Section 4.3, where another scheme is used for this purpose. For the distance computation, the projected coordinate values `lat_m` and `lon_m` are used.

Let us discuss in detail each step:

Step 1: Key Generation

CKKS uses a private/public key pair, which is generated when the first query is triggered:

```
keys = cc.KeyGen()
```

Step 2: Encryption

The value that we want to encrypt is copied into a vector of size N, occupying each available slot. This is done because, when the same value is encoded in N slots inside the encrypted vector, N parallel operations can be performed on the encrypted data in a single homomorphic operation. In our case, this is important because we want to compute the distance between `val` (the user's coordinate), and each of the N candidate places from the database.

Among the tested categories, the category with the highest number of samples has $N' = 15$. But, the OpenFHE library requires the number of slots to be a power of two. Therefore, we set $N = 16$. However, the value of N is adjusted to the number of samples in the category. The same value of $N = 16$ is also used for $N' = 12$, $N' = 13$ and $N' = 14$, as they all have the number 16 as the next multiple of 2 greater than them. For this reason, in the results section, it can be observed that, while N ranges in the interval [12,15], the performance remains constant.

A sample piece of code showing how encryption for CKKS works can be observed below:

Algorithm 1 CKKS Broadcast Encryption

1: `N` $\leftarrow$ `128`
2: `plaintext_vector` $\leftarrow [\texttt{val}, \texttt{val}, \ldots, \texttt{val}] \in \mathbb{R}^N$
3: `plaintext_prepared` $\leftarrow$ Encode_Polynomial(`plaintext_vector`, Δ)
4: `ciphertext` $\leftarrow$ Encrypt(`plaintext_prepared`, `pk`)
5: `ciphertext_serialized` $\leftarrow$ Encode_B64(`ciphertext`)
6: **return** `ciphertext_serialized`

The vector of plaintext values is transformed into a polynomial and scaled by the factor Δ to control numerical precision during encrypted computation.

The resulting polynomial is then encrypted using the public key. The result of the encryption is serialized in Base64 format, so that it is ready for transmission over the standard network protocols.

Step 4: Server's Evaluation

One important aspect is that the operations are performed between ciphertexts (the user's coordinates) and plaintext values (representing the candidate points from the OpenStreetMap dataset). CKKS supports this type of operation. Therefore, there is no need to encrypt the candidates. However, they still need to be encoded in a CKKS-compatible format. The distance computation is then performed in the same manner as in the plaintext case, using Euclidean distance:

$$\text{dist}^2\left(\text{Enc}(x), \text{Enc}(y), x_i, y_i\right) = \left(\text{Enc}(x) - x_i\right)^2 + \left(\text{Enc}(y) - y_i\right)^2 \tag{4.9}$$

Another important aspect is that the use of the Euclidean distance is possible because the latitude and longitude values are projected and therefore represented in meters instead of degrees. This transformation is discussed in detail in Chapter 5.

After the distances are computed, the function returns the encrypted results. Normally, since the client requests only the closest point, the next step would be to choose the argument of the minimum value among the computed distances. However, CKKS does not directly support comparison operations. Therefore, the FHEW scheme is used for this task, through a scheme switching mechanism.

Algorithm 2 OpenFHE CKKS Evaluation of Encrypted Squared Distances

// Load cryptographic context and keys
`cc` ← LoadCrypto(`crypto_context`, `keys`)

// Fetch POI coordinates from the database
`candidates` ← GetCandidatesByCategory(`category`)
`no_candidates` ← |`candidates`|

// Deserialize encrypted query coordinates
`enc_client_x` ← Deserialize(`encrypted_qx`)
`enc_client_y` ← Deserialize(`encrypted_qy`)

// Encode POI coordinates as plaintext polynomials
`py_plain` ← CKKS_Encode(`candidates_lat`)
`px_plain` ← CKKS_Encode(`candidates_lon`)

// Compute squared Euclidean distances homomorphically
`diff_y` ← Subtract(`enc_client_y`, `py_plain`)
`diff_x` ← Subtract(`enc_client_x`, `px_plain`)
`diff_y2` ← Multiply(`diff_y`, `diff_y`)
`diff_x2` ← Multiply(`diff_x`, `diff_x`)
`dist2_ct` ← Addition(`diff_y2`, `diff_x2`)
`dist2_ct` = Enc$\big([\texttt{dist}_0^2, \ldots, \texttt{dist}_{\texttt{actual_n}-1}^2, \texttt{overflow}, \ldots]\big)$

Serialize and return the encrypted result
`dist2_ct_b64` ← Base64Encode(Serialize(`dist2_scaled`))
return {`dist2_ct_b64`, `candidates`, `metrics`}

4.3. FHEW: Fastest Homomorphic Encryption in the West

4.3.1. Scheme Overview

FHEW [6] is a Fully Homomorphic Encryption scheme that addresses the latency added by bootstrapping in classical FHE schemes. By combining gate evaluation and bootstrapping into a single step, the bootstrapping procedure is reduced from several minutes to a few seconds only, while still allowing computations of unlimited depth.

4.3.2. Computational Properties Used

Comparison

To compare two encrypted points, Enc($distA$) and Enc($distB$), a subtraction is performed between the two distances:

$$c = Enc(\text{dist}_A) - Enc(\text{dist}_B) \tag{4.10}$$

The sign of the resulting value (c) is determined through the bootstrapping procedure.

The minimum is found by comparing pairs of candidates: (candidate 1 with candidate 2), (candidate 3 with candidate 4), ..., (candidate $n-1$ with candidate n). This approach is used to preserve the number of comparisons to $n-1$ and the sequential depth to $\log_2 N$. It needs $\log_2 N$ rounds of choosing the minimum between pairs, and at each round, half of the candidates are chosen to continue to the next round. This approach is better than a sequential one because it does not increase the depth, and therefore it does not increase the noise as much.

However, all these implementation aspects are handled internally by the FHEW library, and for the implementation of this thesis, an API was called to find the minimum argument among the candidates.

Nevertheless, discussing and understanding these aspects is fundamental because it directly imposes some constraints on how the data, namely the coordinates, must be handled. The most important constraint is that all input values of the comparison operation must be in the range [-1, 1]. For this reason, the real squared distances are scaled by the factor $\delta = 5 \times 10^{-9}$. After the scaling, they fall in the interval [0, 0.5].

4.4. Scheme Switching: From CKKS to FHEW

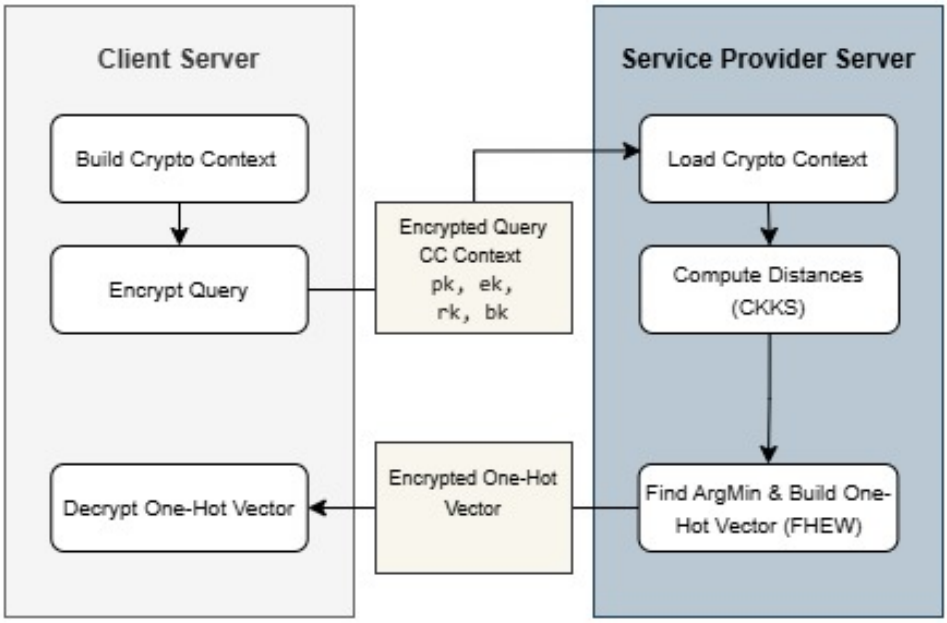


Figure 4.1: Scheme Switching Pipeline

The computation of the distance between each candidate point and the user coordinates is done using the CKKS scheme, but CKKS is not capable of doing comparison operations. For this reason, the FHEW scheme was used for the comparison. The switch between encryption schemes is possible, and the concept will be addressed in this section.

While FHEW is a fully homomorphic encryption scheme which could have been used from the beginning for both comparison and distance computation, which requires multiplications and subtraction, it is slower than CKKS for this specific task (distance computation). Therefore, the chosen solution was to combine them.

The crypto context is created on the client side and holds all the parameters used to set the schemes and the public evaluation keys. It is created only when the OpenFHE

runtime is firstly initialized, and it is used in later queries. The context is created as a CKKS context, where the specific parameters are set, and later it is extended with features such as scheme switching capabilities to transition to FHEW and the FHE feature that enables bootstrapping. Therefore, one single context is used for both schemes.

The result is returned as a one-hot encoded vector. That is because it is aimed to preserve the privacy of the destination of the user, namely which of the candidate locations is the closest to her/him.

4.5. TFHE: Fast Fully Homomorphic Encryption over the Torus

4.5.1. Scheme Overview

Fast Fully Homomorphic Encryption over the Torus, abbreviated as TFHE [7], is a fully homomorphic encryption scheme. The scheme supports fast bootstrapping, which is exact, without adding more error when refreshing the noise. It also enables an unlimited number of sequential gate evaluations.

The most important operation of TFHE is the Programmable Bootstrapping (PBS) operation. The operation refreshes the ciphertext and evaluates a function at the same time. Plaintext values are encoded as elements of T and then encrypted under Learning with Errors (LWE). The scheme has a strong post-quantum security guarantee.

4.5.2. Computational Properties Used

Addition

The homomorphic addition is the least expensive operation in the TFHE scheme because it does not require bootstrapping. The reason is that the noise grows linearly when adding two ciphertexts.

For TFHE, addition on ciphertexts is performed in the same way as addition on plaintexts:

$$Enc(a + b) = Enc(a) + Enc(b) \tag{4.11}$$

Programmable Bootstrapping (PBS)

Programmable bootstrapping combines two operations in one:

- refreshes the ciphertext noise
- evaluates a univariate function that is encoded as a lookup table

As a consequence, non-linear operations (e.g. rounding, absolute value) are possible in a single pass.

Multiplication

TFHE does not support multiplication directly. Instead, the multiplication is done using a PBS, through a lookup table. The syntax for using the multiplication is the same as in plaintext, but in reality, each multiplication on encrypted data triggers a PBS.

Comparison Operations

TFHE directly supports encrypted comparison by using a lookup table, which is evaluated through Programmable Bootstrapping (PBS). The comparisons are performed pairwise, not sequentially, as this does not increase the sequential depth.

Key Compression

The Concrete library has the option to serialize the keys in a compressed form, which reduces the key size by about 36 times. This reduces the latency of the setup stage, during which the client sends the keys to the server.

4.6. Location Compliance Verification: Theoretical Protocol

This protocol uses threshold CKKS multiparty homomorphic encryption over the following parties:

- the employee
- the company

4.6.1. Parties and Trust Model

The entities that participate in the protocol are:

- **Employee:** holds a secret key share with which they encrypt their current location.
- **Company:** holds the second secret key share with which it encrypts the location policy (home address, office address, country, and radius).
- **Service Provider:** receives the cryptographic context and the public and evaluation keys, but never the secret key. It has access to none of the plaintext addresses of the employee or the employer.

The privacy guarantees are that the employee does not know the company's location policy, the company does not know the employee's current location, and the service provider knows none of the locations. Neither the employee nor the company can decrypt the final status alone, as both need to participate in the decryption.

4.6.2. Key Setup

The key setup protocol is based on the threshold variant of the CKKS scheme from the OpenFHE extension for multiparty computation. The process is as follows:

1. The employee generates the first key pair `(pk_initial, sk_employee)` using the standard key generation procedure from CKKS.
2. The employee sends the public key `pk_initial` to the company.

3. The company extends the key received from the employee using `MultipartyKeyGen`($pk_initial$). The resulting keys are `(pk_joint, sk_company)`. Therefore, the joint public key `pk_joint` is formed using both secret keys and will be used for all encryptions.

4. The company sends the `pk_joint` key back to the employee.

5. Evaluation keys are generated independently by each party and then combined through `MultiAddEvalMultKeys` into a single joint evaluation key. This key is added to the cryptographic context, which is forwarded to the service provider.

4.6.3. Encryption Phase

After receiving the pk_{joint} key, both parties can start encrypting their data. The company encrypts the employee's declared home address, the office area, the radius, and the country of residence when the company is registered or when an edit is made. Each zone (office/home/country) is represented by four boundaries.

The employee encrypts their current latitude and longitude as two scalar ciphertexts under pk_joint.

4.6.4. Server-Side Computation

The service provider receives the ciphertexts and then computes the encrypted differences or margins for each zone:

$$margin_{\mathrm{N}} = \texttt{EvalSub}\,(\mathrm{enc}(\mathrm{boundary_north}), \mathrm{enc}(\mathrm{lat})) \tag{4.12}$$

$$margin_{\mathrm{S}} = \texttt{EvalSub}\,(\mathrm{enc}(\mathrm{lat}), \mathrm{enc}(\mathrm{boundary_south})) \tag{4.13}$$

$$margin_{\mathrm{E}} = \texttt{EvalSub}\,(\mathrm{enc}(\mathrm{boundary_east}), \mathrm{enc}(\mathrm{lon})) \tag{4.14}$$

$$margin_{\mathrm{W}} = \texttt{EvalSub}\,(\mathrm{enc}(\mathrm{lon}), \mathrm{enc}(\mathrm{boundary_west})) \tag{4.15}$$

If a margin is positive, it means that the employee is inside that boundary for the corresponding zone. To be considered inside a zone, the employee must have all margins positive.

4.6.5. Threshold Decryption

There are three phases in the decryption process:

1. **Employee:** The employee uses the key `sk_employee` to partially decrypt each of the twelve margin ciphertexts, resulting in twelve partially decrypted values.

2. **Company:** The company uses the key `sk_company` to partially decrypt the original ciphertexts received from the server, resulting in another twelve partially decrypted values.

3. **Fusion:** The company receives the partial decryption results from the employee, combines them with its own partial decryption results, and calls `MultipartyDecryptFusion` on each pair of values. The final result consists of the plaintext values.

4.6.6. Compliance Evaluation

In the initial implementation, after the margins are decrypted, the result is evaluated for each zone. For a given zone, the employee is considered to be inside it if and only if all margins are positive. This process is repeated for each zone: home, country, and office.

Later, it was taken into account that encrypted computations may introduce errors. Therefore, a threshold was added, and the margins must be greater than $-\epsilon$.

4.7. Detection of Account Takeover

4.7.1. eXtreme Gradient Boosting (XGBoost)

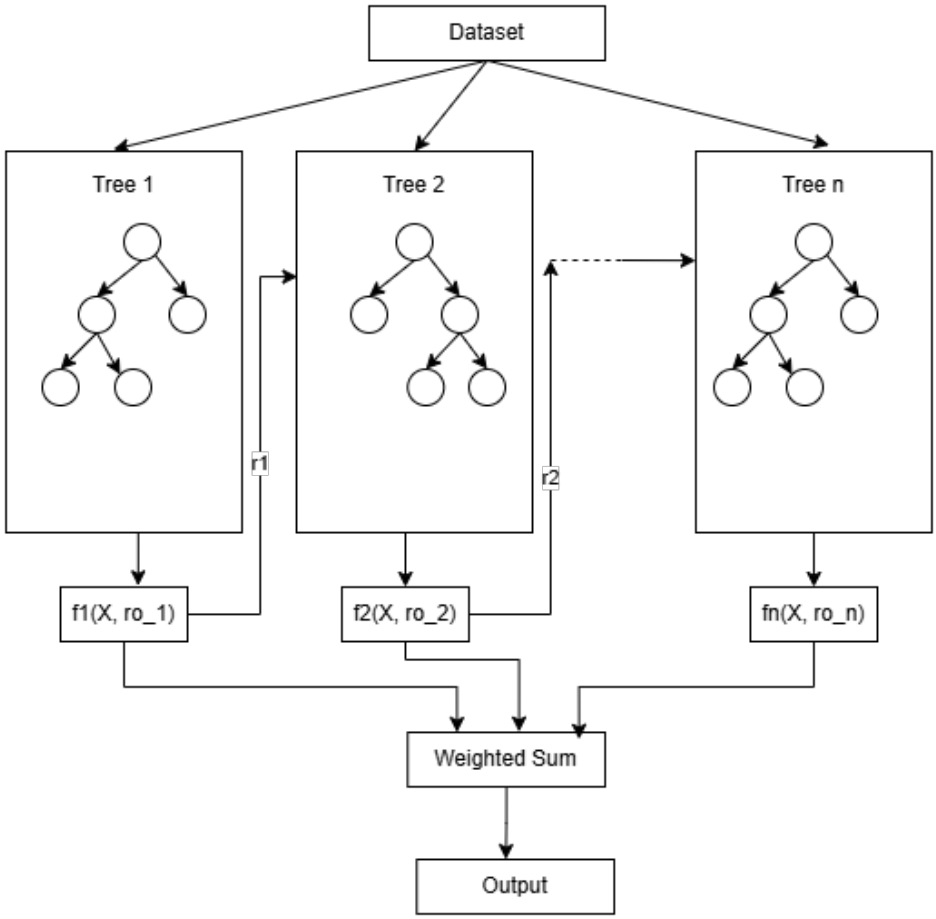


Figure 4.2: XGBClassifier Model

The library used for the RBA module was XGBoost, which stands for Extreme Gradient Boosting. The XGBClassifier model provided by this library was used.

The classifier can be used for both binary and multiclass classification. It combines multiple decision trees, which are created sequentially. The first tree is created and makes a prediction, after which the model computes the error made by this tree. A second tree is then built and trained to correct these errors, and the process continues in the same way. Therefore, the general rule is that each newly added tree must improve the performance of the previous ones. After each tree makes a prediction, the results are combined to produce the final result. It uses both Lasso (L1) and Ridge (L2) regularization to reduce the model complexity and therefore prevent the overfitting.

The residual r_i, used for training the tree $i+1$, is the gradient of the loss function, negated, with respect to the prediction done by the previous ensemble:

$$r_i = -\left[\frac{\partial L\left(y_i, F(x_i)\right)}{\partial F(x_i)}\right], \tag{4.16}$$

The partial results are combined using a weighted sum. The output of each tree is multiplied by the same learning rate:

$$F_T(x) = F_0(x) + \sum_{t=1}^{T} \eta f_t(x), \tag{4.17}$$

where:

- $F_0(x)$ - the initial prediction
- $f_t(x)$ - the score returned by tree t;
- η - the learning rate

The most important parameters of this model are:

- `n_estimators`: specifies how many trees the model should have
- `learning_rate`: controls how much each tree contributes to the final result
- `max_depth`: specifies the maximum depth that each tree can have
- `gamma`: controls whether the model is allowed to create a new split in the tree. For each possible split, the model computes how much it reduces the loss function. If the reduction is greater than `gamma`, the split is kept
- `reg_alpha` (L1 regularization): adds penalties for leaves with large weights to reduce overfitting. It can result in setting the weights of some leaves to zero
- `reg_lambda` (L2 regularization): adds penalties for leaves with large weights to reduce overfitting. It reduces the size of the leaf weights but cannot set them to zero
- `objective`: defines the learning task and the loss function used by the model
- `scale_pos_weight`: is useful for imbalanced datasets, as it gives greater importance to the minority class
- `early_stopping_rounds`: is used during training to stop the process when the evaluation metric no longer improves for a specified number of rounds

4.7.2. Evaluation Metrics for Imbalanced Classification

As the dataset is highly imbalanced, measuring accuracy would be misleading. This is because accuracy can be very high even if the model performs well only on the majority class. Therefore, accuracy is not relevant for RBA, and the chosen metrics are recall and ROC-AUC.

Recall

The most important metric for measuring the performance of the RBA model is recall. Recall measures how many real account takeovers are actually detected and is computed as:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4.18}$$

where:

- TP - the number of true positives
- FP - the number of false positives
- TN - the number of true negatives
- FN - the number of false negatives

Accuracy

Accuracy measures how many of the total predictions are correctly classified.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.19}$$

ROC-AUC

ROC-AUC measures how well the model can distinguish between real attacks and normal login attempts for different classification thresholds.

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)} \tag{4.20}$$

$$FPR(t) = \frac{FP(t)}{FP(t) + TN(t)} \tag{4.21}$$

$$\text{ROC-AUC} = \int_0^1 TPR(FPR), dFPR \tag{4.22}$$

where:

- t - classification threshold

4.7.3. Privacy Model

The main privacy concern with Risk-Based Authentication is that it uses sensitive data such as location, travel schedules, data about the device, and so on. To protect the user from the honest-but-curious server, another variant is proposed:

- the model is trained on plaintext;
- the inference using the user data is performed on encrypted data.

This is possible using the Concrete-ML library, which implies that one more variant must be trained, specially prepared for HE inference.

4.8. Communication Protocol between Client and Server

While the client and the service provider are two independent services, they need to communicate so that the client can transmit the user's encrypted coordinates, together with the public and evaluation keys, to the server. The server then returns the result after performing the computation. This communication requires a protocol that is efficient and capable of transmitting large Homomorphic Encryption ciphertexts and keys (which are known to have large sizes). Therefore, the protocol selected for providing the above mentioned capabilities is gRPC.

Overview of gRPC

gRPC is a Remote Procedure Call framework that was developed by Google. It uses Protocol Buffers for the interface definition language and serialisation. It runs over HTTP/2. While the REST protocol transmits data in a JSON format, which is human-readable, over HTTP/1.1, gRPC serialises it in a binary format and can transmit multiple calls over a single TCP connection. As a result, the latency and payload are much lower.

The `.proto` file is the equivalent of a contract/interface, and it defines services and data structures.

The services define the remote procedures that the server should implement and that can be called by the client.

RPCs can be of multiple types, from which unary RPC and client-streaming RPC were used. In unary RPC, the client sends one request and receives one response. This is used for almost all service requests between the client and the server. In client-streaming RPC, the client sends multiple requests that replace one large message. In this thesis, it was used to send the keys from the client to the server, as they are very large.

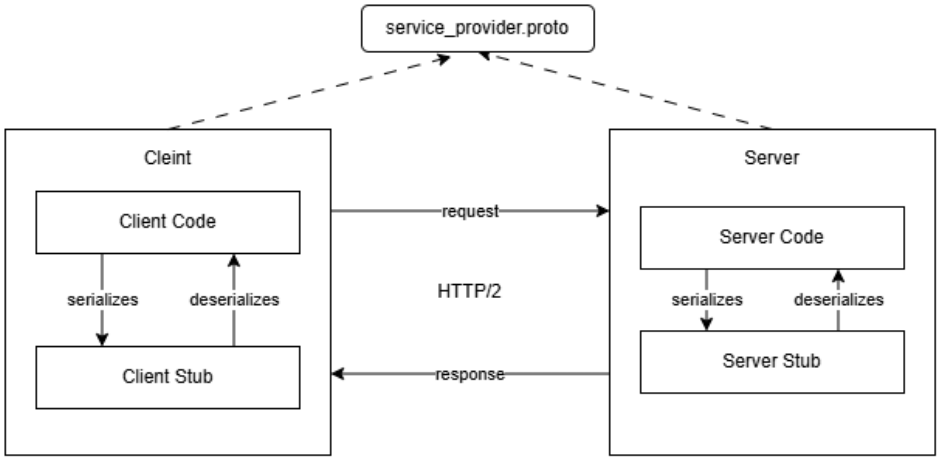


Figure 4.3: gRPC Protocol

Chapter 5. Detailed Design and Implementation

5.1. System Design and Architecture

5.1.1. Conceptual Architecure

The system is composed of three main components: the User Interface (UI), the client app, which is a backend service, and the service provider app. The users communicate through the UI with the client app, which securely handles their data. However, all the heavy computations and access to databases are done by the service provider.

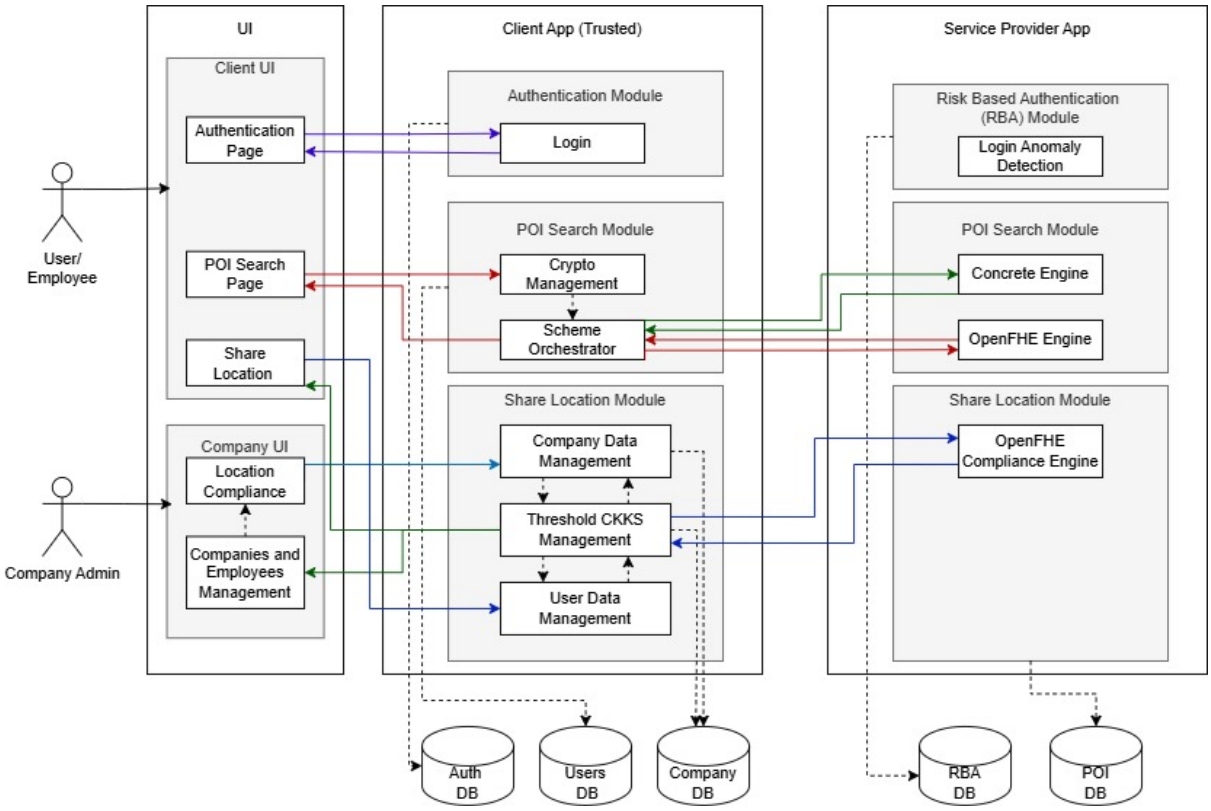


Figure 5.1: Conceptual Diagram

5.1.2. Deployment Diagram

Some of the most important libraries used in the development of this thesis were Concrete and Concrete-ML, which are natively compatible with Linux and macOS. Because the OS of this machine is Windows, a design choice in developing this system was to deploy it in Docker, such that it both ensures compatibility between the OS and the

libraries and provides better isolation. It also helps in reproducing the runtime environment on other machines.

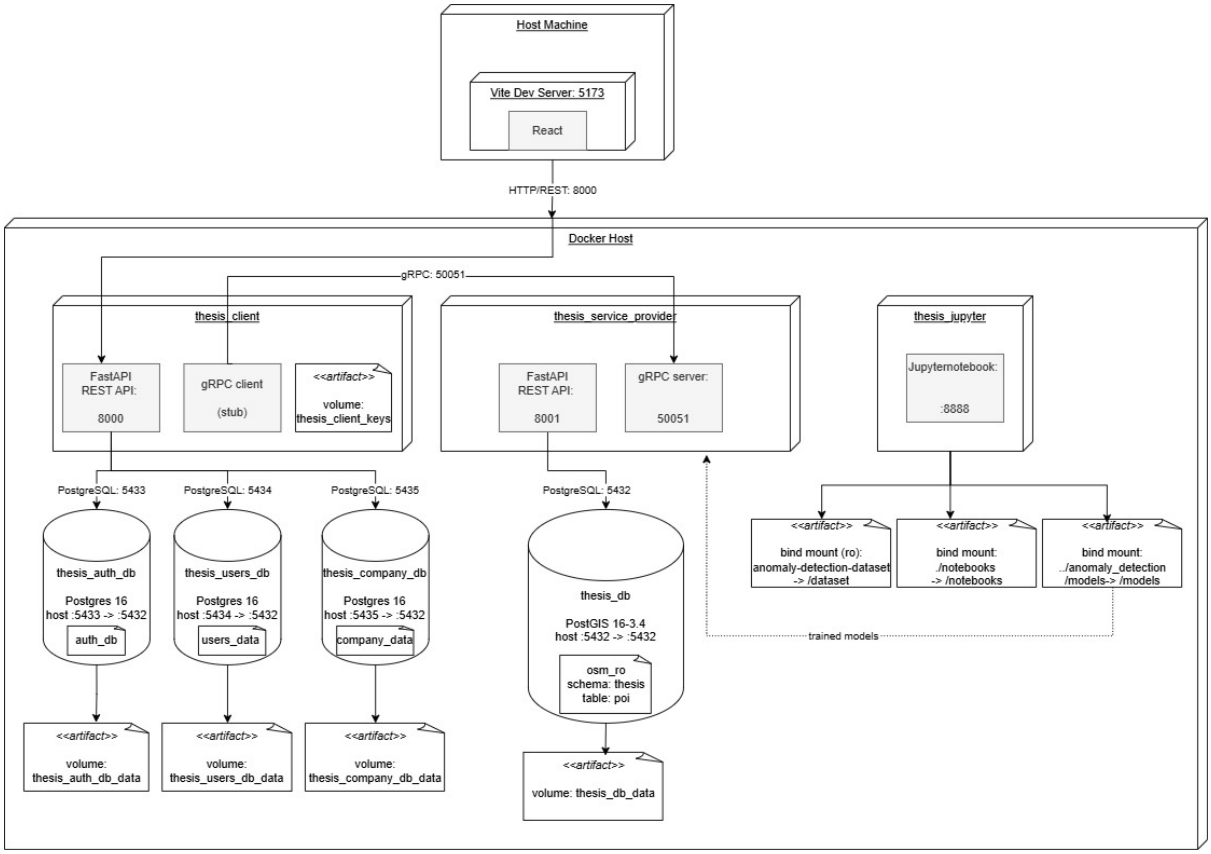


Figure 5.2: Deployment Diagram

5.2. Preprocessing of OpenStreetMap (OSM) Data

The Points of Interest (POI) dataset was derived from OpenStreetMap, a public and freely available global geographic database maintained by volunteers and organizations. Only data from Cluj-Napoca was downloaded using the Overpass API.

The initial dataset was composed of multiple tables, including `nodes`, `ways`, `way_nodes`, `relations`, `tags`, etc.. Using an SQL script, the relevant data was extracted and merged into a single table containing only the required information:

- the initial ID of the node, representing the point of interest in the database
- name
- category
- latitude
- longitude
- city

The latitude and longitude were originally represented in degrees, but it was applied a transformation to transform them in planar Cartesian coordinates expressed in meters, stored as lat_m and lon_m. The conversion was performed using Web Mercator formula:

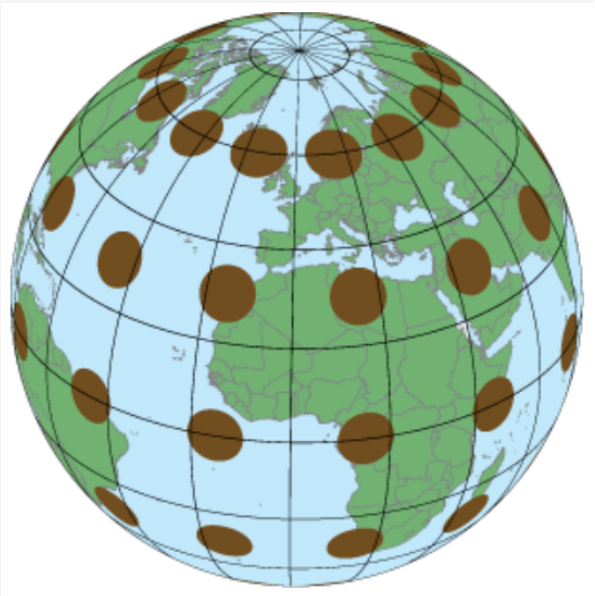
$$\text{lon_m} = R\lambda \tag{5.1}$$

$$\text{lat_m} = R \ln\left(\tan\left(\frac{\pi}{4} + \frac{\varphi}{2}\right)\right) \tag{5.2}$$

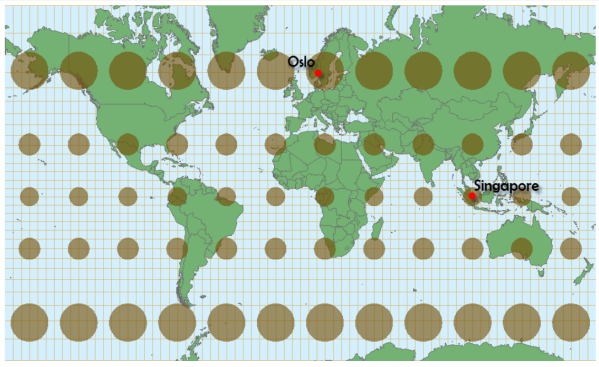
where:

- $R = 6\,378\,137\,\text{m}$ - the Earth radius used by the Web Mercator projection
- λ - the longitude converted from degrees to radians
- φ - the latitude converted from degrees to radians

As a note, Web Mercator is essentially the projection of the spherical globe onto a 2D map.



(a) Spherical representation of the Earth



(b) 2D map

Figure 5.3: Transformation from the spherical Earth to a 2D map using the Web Mercator projection.

This transformation allows the Euclidean distance between locations to be computed directly. It was observed that all locations in Cluj-Napoca shared the same prefix: "2,617" for longitude and "5,891" for latitude. To reduce memory usage and improve processing speed, these common prefixes were removed from the `lon_m` and `lat_m` values.

$$\text{lon_m} = \text{lon_m} - 2{,}617{,}000 \tag{5.3}$$

$$\text{lat_m} = \text{lat_m} - 5{,}891{,}000 \tag{5.4}$$

In order to improve the query efficiency, PostGIS was used. PostGIS is an extension of PostgreSQL, which adds spatial data types and corresponding functions and indexing for querying the points faster. Each of the points was stored as a EPSG:3857 POINT, in a new column called "geom" and a spatial GiST was created.

The final shape of the table is shown below. The city column is not displayed due to the inability to fit such a wide table on the page, but its value is Cluj-Napoca for every row:

id	name	category	lat	lon	lat_m	lon_m	geom
2329184457	Anticariat Academic	books	46.77183690020694	23.5934157	13911.2425694242	9407.02179802628	POINT (2626407.0217980263 5904911.242569424)
8043819440	terrawell.ro	chemist	46.77186990020694	23.593542199999998	13916.606149043	9421.10371361161	POINT (2626421.1037136116 5904916.606149043)
3708434798	Magic Natura	health_food	46.771931900020693	23.593806299999997	13926.6831863075	9450.50319112977	POINT (2626450.50319113 5904926.683186308)
2329181777	Pell Art	clothes	46.771959500206904	23.5938896	13931.1690969476	9459.77610471332	POINT (2626459.7761047133 5904931.169096948)
2852526202	Mark Lighting	electrical	46.772075500206896	23.594359999999998	13950.0229494227	9512.14079318242	POINT (2626512.1407931824 5904950.022949423)
2329186699	Orange	mobile_phone	46.77241910020683	23.5944495	14005.8695990071	9522.10388760827	POINT (2626522.1038876083 5905005.869599007)
9709196246	Grain Lab	photo	46.77229470020685	23.593806599999997	13985.6503401408	9450.53658697708	POINT (2626450.536586977 5904985.650340141)
1583747943	CEC Bank	bank	46.772599200206805	23.5946983	14035.142097529	9549.80017691758	POINT (2626549.8001769176 5905035.142097529)
2177295925	CEC Bank	atm	46.77282990020673	23.594649199999996	14072.6389946081	9544.33438991942	POINT (2626544.3343899194 5905072.638994608)
4128187386	Croitorie	tailor	46.77314250020668	23.5933881	14123.4477902427	9403.94938008022	POINT (2626403.94938008 5905123.447790243)
736625363	After 8	nightclub	46.77262860020679	23.5932095	14039.920626848	9384.06771902461	POINT (2626384.0677190246 5905039.920626848)
736625359	Albă ca Zăpada	kindergarten	46.772983000206736	23.5926642	14097.5232399506	9323.36520069512	POINT (2626323.365200695 5905097.523239951)

Table 5.1: Sample of the processed OpenStreetMap POI dataset

The database in which the POI dataset is stored is a PostgreSQL 16 relational database. The data is stored under the `thesis` schema in the `poi` table. The database is hosted inside a Docker container, and the data was imported from the preprocessed CSV file.

In the simulations, only six categories were selected from the dataset. This is because, when using TFHE from the Concrete library, the value of N must be known in advance when compiling the circuit.

Category	***Count in dataset***	***Circuit Size*** N
Hospital	1	1
School	5	5
Stationery	12	12
`Post_Office`	13	13
Cosmetics	14	14
Gallery	15	15

Table 5.2: Selected categories

5.3. Finding the Nearest Point of Interest: CKKS + FHEW

As explained previously in Chapters 4.2 and 4.3, neither CKKS nor FHEW alone can solve the task of computing the distances between points and selecting the minimum. Therefore, they are combined.

5.3.1. Parameters and Key Setup

The client creates an `OpenFHESSClientRuntime` singleton when it is called for the first time (`openfhe_ss_runtime_instance.py`). The security context created is a CKKS context, and the cryptographic parameters are as follows:

Batch Size

The CKKS scheme uses polynomial rings. This allows multiple values to be packed simultaneously into one ciphertext. The number of slots in the ciphertext represents the batch size.

Therefore, the client writes the user coordinates into 128 slots inside the ciphertext it sends. Similarly, the server packs the coordinates of the 128 candidates into the slots. This allows the computation of the distance between the client coordinates and each POI to be performed in parallel.

The number 128 was chosen because the category with the highest number of samples has (N' = 122). However, the OpenFHE library requires the number of slots to be a power of two. Therefore, we set N = 128. As not all categories have 122 samples, the unused slots are padded.

Multiplicative Depth

Multiplicative depth represents the maximum number of sequential multiplications that can be performed before the resulting ciphertext becomes too noisy.

During the computation of the Euclidean distance, two values need to be squared, which is equivalent to multiplying each value by itself. Therefore, two multiplications are needed, each consuming one level of multiplicative depth.

Additionally, the final distance is scaled, which means multiplying the result by a scaling factor, therefore consuming one more level of multiplicative depth.

Switching from CKKS to FHEW triggers the transformation of floating-point numbers into integers. The approximation is performed using a polynomial, but a precise approximation would require a high-degree polynomial. The degree is chosen internally by the OpenFHE library.

Therefore, only the maximum multiplicative depth can be set. We know that at least three levels are used for the multiplications discussed above, and a multiplicative depth of 52 was experimentally found to provide a good precision.

Scaling Modulus

While CKKS works with floating-point numbers, the underlying ring is defined over integers. Therefore, the scaling factor indicates how precisely the library should perform the transformation. A larger scaling factor results in higher precision. Additionally, after each multiplication in CKKS, the result needs to be rescaled to keep it bounded.

Ring Dimension

The ring dimension is the degree of the polynomial used to store the CKKS data. A larger ring dimension results in better security, but also in larger ciphertexts and, consequently, slower computation. The chosen ring dimension is 2^{14}.

FHEW security

The security level for FHEW is set to `STD128`, which means 128-bit post-quantum security. Therefore, to break the ciphertext, an attacker would need 2^{128} operations.

The client:

- generates a CKKS key pair (secret key `sk` and public key `pk`) by calling `EvalSchemeSwitchingSetup`
- generates the FHEW bootstrapping keys using `BTKeyGen`
- links the CKKS and FHEW keys through `EvalSchemeSwitchingKeyGen`

The following components are therefore needed to be sent to the server:

- the cryptographic context with the parameters discussed above
- the public key
- the evaluation multiplication keys
- the rotation keys, needed to rotate the slots in the ciphertext when the comparison is performed
- the encrypted coordinates
- the plaintext category

5.3.2. Server-Side Computation

When the server receives the request, it also finds in plaintext the name of the category it must compute for. Let us call this category (C). Therefore, it fetches the candidates from category (C) from the database and encodes them into two CKKS plaintext vectors, one for each coordinate.

```
py_plain = cc.MakeCKKSPackedPlaintext(py_vals)
px_plain = cc.MakeCKKSPackedPlaintext(px_vals)
```

After this, the evaluation keys and the cryptographic context are loaded. Then, the encrypted coordinates are also deserialized:

```
ct_qy = _load_ciphertext(encrypted_qy)
ct_qx = _load_ciphertext(encrypted_qx)
```

To compute the distances, the Euclidean distance formula can be applied directly to the encrypted data:

```
dy = cc.EvalSub(ct_qy, py_plain)
dx = cc.EvalSub(ct_qx, px_plain)

dy2 = cc.EvalMult(dy, dy)
dx2 = cc.EvalMult(dx, dx)

dist2_ct = cc.EvalAdd(dy2, dx2)
```

Because not all categories have a number of candidates equal to the batch size, padding is enforced. The padding is done in such a way that it does not provide a synthetic distance to the user that is smaller than the true distances and therefore does not alter the result.

5.3.3. Scheme Switching

In the next step, the server needs to find the minimum, so it switches the scheme to FHEW.

```
[ct_min, ct_one-hot] = EvalMinSchemeSwitching(ct_v, pk, 128, 128)
```

5.3.4. Client-Side Decryption

The client receives the one-hot-encoded vector containing the minimum and decrypts it using the secret key.

5.3.5. Pipeline

5.4. Finding the Nearest Point of Interest: TFHE

5.4.1. Coordinate Preprocessing

TFHE operates on integers whose number of bits is determined at compile time. Because of the bootstrapping operation, integers are limited to a maximum of 8 bits. Since our coordinates are floating-point numbers, they must be rounded and fitted within the appropriate interval.

The distance is a result of the Euclidean formula, which is:

$$\text{dist} = (x_1 - x_2)^2 + (y_1 - y_2)^2 \tag{5.5}$$

We impose the constraint that dist must be represented using at most 8 bits, which means that:

$$\text{dist} < 256 \ (= 2^8) \tag{5.6}$$

Therefore, in the worst-case scenario, the distance should be:

$$\text{dist} = (11-0)^2 + (11-0)^2 = 242 \tag{5.7}$$

Many combinations of coordinate values can trigger a result close to 256, but this scenario was considered to determine the maximum value that one coordinate can have, and the answer is 11. As a consequence, we need to map the coordinates inside the interval $[0, 11]$. The point of interest with the highest value had:

$$\text{lon_m} = 2{,}641{,}200 \tag{5.8}$$

which, after shifting the reference (removing the prefix), becomes:

$$\text{lon_m} = 2{,}641{,}200 - 2{,}617{,}000 = 24{,}200 \text{ m} \tag{5.9}$$

Mapping it into the interval $[0, 11]$ would require the following factor:

$$\text{scaling factor} = \frac{24{,}200}{11} = 2{,}200 \tag{5.10}$$

Therefore, all the coordinates are divided by a factor of 2,200 metres and then rounded to the nearest integer:

```
quant_config = QuantizationConfig(scale_divisor=2200)

...

qx = quantize_metric_coordinate(float(longitude), quant_config) - ORIGIN_X_Q

qy = quantize_metric_coordinate(float(latitude), quant_config) - ORIGIN_Y_Q
```

5.4.2. Circuit Design

Because each category had a different number of samples/candidate points (N), a Concrete circuit was created for each value of N.

```
SUPPORTED_N = [1, 5, 12, 13, 14, 15]
```

The circuits are dynamically built for each N by the function `def _make_circuit_fn(n: int)`. Each circuit has the following signature:

```
circuit(qx_rel, qy_rel,
        p1x, p1y, p1_id,
        p2x, p2y, p2_id,
        ...,
        pNx, pNy, pN_id)
```

where:

- `qx_rel`, `qy_rel`: the encrypted coordinates of the user
- pi`x`, pi`y`: the plaintext coordinates of point of interest i

- pi_id: the plaintext identifier of point of interest i

To find the minimum, a linear scan is performed across all points. The minimum is initially set to the squared distance between the user and the first point of interest. An important note is that the minimum identifier must be added to an encrypted zero so that it is brought into the encrypted domain. Otherwise, it would create a mismatch at the first scan, when updating the min_id, as pi_id would need to be encrypted. This is a security privacy, as the server is not supposed to know what index will hold the minimum value in the end.

```
def circuit(qx_rel, qy_rel,
            p1x, p1y, p1_id,
            ...,
            pNx, pNy, pN_id):

    encrypted_zero = qx_rel - qx_rel

    dx = qx_rel - p1x
    dy = qy_rel - p1y
    min_dist = dx * dx + dy * dy

    min_id = p1_id + encrypted_zero

    for i in range(2, N + 1):
        dx = qx_rel - pix
        dy = qy_rel - piy
        d = dx * dx + dy * dy

        is_new_min = d < min_dist

        min_dist = np.where(
            is_new_min, d, min_dist
        )

        min_id = np.where(
            is_new_min,
            pi_id + encrypted_zero,
            min_id
        )

    return min_id, min_dist
```

As complexity, the circuit performs (N-1) iterations, where each iteration requires 3 PBS operations: one for the comparison, one for the `min_dist` update, and the other for the `min_id` update. In total, 3(N-1) operations are needed.

These circuits are compiled before the application is started by the script `client_app/tools/compile_distance_circuit.py`. Compilation is done using `fhe.Compiler`, and it produces two artifacts:

- `server_nN.zip` - this is the compiled circuit for the server and is loaded by the service provider at startup
- `client_specs_nN.bin` - this is the client specification file and is used to generate keys, encrypt inputs, and decrypt results

To determine the bit width of every intermediate value, Concrete traces the circuit over representative inputs, which are as follows:

- **Type 1:** The user is at position $(0, 0)$, and all the points are at position $(11, 11)$. This type sets the maximum distance value

- **Type 2:** The user is at position $(11, 11)$, and all the points are at position $(0, 0)$. This type sets the maximum value for the encrypted user coordinates

- **Type 3:** The user is at position $(5, 5)$, the last POI is at $(5, 5)$, and all POIs except the last are at $(0, 0)$. This type makes `min_id` equal to N, which is the maximum possible value, and `min_dist` equal to 0, which is the minimum possible value

These scenarios allow Concrete to allocate the correct bit widths, as it knows all the extreme values, both minimum and maximum.

The circuit is associated with a noise parameter, `p_error`, which represents the probability that a PBS operation is incorrect. Therefore, the probability that the circuit is incorrect is:

$$P(\text{circuit error}) \approx 1 - (1 - p_{\text{error}})^{\text{no. PBS operations}} \tag{5.11}$$

More specifically, for `p_error` $= 10^{-5}$ and PBS $= 3(N-1)$, the equation becomes:

$$P(\text{circuit error}) \approx 1 - \left(1 - 10^{-5}\right)^{3(N-1)} \tag{5.12}$$

5.4.3. Key Compression

For Concrete, the evaluation keys are the largest data that need to be sent from the client to the server. If compression was not used, the size of the key for (N = 5) was 5 GB. If compression is used, the size is reduced approximately 40 times. Initially, the HTTP protocol was used to send the keys instead of gRPC, but it failed to transmit them due to the size of the evaluation keys.

5.4.4. Client-Server Protocol

Step 1: Selection of the Circuit Size

The client sends a query to the service provider to find the number of POIs, such that it will know what circuit to select. This request is sent over HTTP.

```
GET /service/poi/category-count?category=<category>
```

Based on the returned result, the client selects the size of the circuit N to be equal to or larger than the number of POIs.

Step 2: Query Preparation, Generation of Keys

The `ConcreteClientRuntime(n)` object is instantiated by the method `ConcreteClientAdapter.prepare_query()`. The object loads the client specification from the file `client_specs_nN.bin`. After this, the client quantizes the user's coordinates and generates the keys with which it will encrypt the coordinates. The keys that need to be transmitted are serialized.

Step 3: Key Transmission

The keys are transmitted over the gRPC protocol. Due to the fact that the keys are still excessively large, sending them in one message would result in a payload that might fail to be transmitted. Therefore, the message is split into 4 chunks and sent sequentially.

The service provider will store the evaluation keys, which will expire after one hour (TTL = 3600 s). This mechanism prevents the keys from being transmitted on every single request made in the same session.

Step 4: Server's Evaluation

The method `ConcreteServerRuntime.run_find_minimum()` inside the service provider loads the corresponding pre-compiled circuits from the file `server_nN.zip`. The server then deserializes the evaluation keys and the encrypted coordinates. The argument list is constructed:

```
[
    encrypted_qx,
    encrypted_qy,
    p1x, p1y, p1_id,
    ...,
    pNx, pNy, pN_id
]
```

Finally, `server.run(*args, evaluation_keys=evaluation_keys)` is called, and it returns the encrypted result, containing `(best_id, best_dist2)`. They are serialized and returned back to the client.

Step 5: Client Decryption

The method `concreteClientAdapter.parse_response()` deserializes the input and decrypts it. The result must be de-quantized into its original form:

$$\text{distance_m} = \sqrt{\text{best_dist2}} \times 2{,}200 \tag{5.13}$$

$$\text{distance_km} = \frac{\text{distance_m}}{1{,}000} \tag{5.14}$$

The final step is the retransmission of the result to the server.

5.4.5. Pipeline

5.5. Location Compliance Verification

5.5.1. Parameters and Key Setup

Because this application is a demo, it was taken into consideration that it should perform faster and be easier to test than a complete, professional application. Hence, the decision was made to generate new public and secret keys only when the container is created.

The process of creating the keys is as follows:

0. The cryptographic context is created:

```
self.cc = self._create_context()
```

1. The client generates its original pair of keys, public and secret, by calling the function `KeyGen()`:

```
user_keys = self.cc.KeyGen()
```

2. The client sends its original public key to the company.

3. The company creates its original pair of keys, public and secret. An important aspect is that it uses the user's public key as input in the generation and calls the function `MultipartyKeyGen()`:

```
company_keys = self.cc.MultipartyKeyGen(
    user_keys.publicKey
)
```

4. The company's public key becomes the joint public key, as it was created last and contains contributions from both parties:

```
self.joint_public_key = company_keys.publicKey
```

5. The evaluation keys are created, and all the keys are saved to disk:

```
self._generate_eval_mult_keys()
self._save_to_disk()
```

The evaluation keys are also generated with contributions from both parties. The user creates its evaluation key first, using its secret key:

```
eval_mult_key_user = self.cc.KeySwitchGen(
    self._user_sk,
    self._user_sk,
)
```

The company receives the evaluation key from the user and calls `MultiKeySwitchGen()` to generate the company's evaluation key:

```
eval_mult_key_company = self.cc.MultiKeySwitchGen(
    self._company_sk,
    self._company_sk,
    eval_mult_key_user,
)
```

The joint evaluation key is created by combining the evaluation keys of the user and the company:

```
eval_mult_key_joint = self.cc.MultiAddEvalMultKeys(
    eval_mult_key_user,
    eval_mult_key_company,
    self.joint_public_key.GetKeyTag(),
)
```

The public key and the evaluation keys are transmitted to the server as soon as they are generated.

One important limitation of this implementation is that it was designed for a single-user scenario. Extending it to multiple users and multiple pairs of user-company parties would require a key management component, as well as scaling the service provider capabilities to support multiple requests, while maintaining performance.

5.5.2. Company Server Computations

The administrator is the only account allowed to create a new company. When the administrator creates a new company, he must define:

- the office coordinates: latitude and longitude
- the name of the company
- the country boundaries: south boundary, north boundary, west boundary, and east boundary
- the radius from the office centre that is supposed to cover the entire building

After creation, the company data is immediately encrypted and stored. Before encryption, a scaling factor of $s = 100$ is applied to all data fields to reduce the size of the data:

```
COORD_SCALE = 100.0
```

As a limitation, the only shape allowed for a country is a bounding box, as defining a custom boundary would be a very complex task outside the scope of this thesis.

```
def create_company(
    self,
    request: CreateCompanyRequest,
    current_user: dict
) -> dict:
    data = request.model_dump()
    enc = openfhe_multiparty_runtime.encrypt_scalar

    data["encrypted_office_lat_m"] = enc(
        request.office_lat_m / COORD_SCALE
    )
    data["encrypted_office_lon_m"] = enc(
        request.office_lon_m / COORD_SCALE
    )
    data["encrypted_office_radius_m"] = enc(
        request.office_radius_m / COORD_SCALE
    )
    data["encrypted_country_north_m"] = enc(
        (request.country_north_m or 0.0) / COORD_SCALE
    )
    data["encrypted_country_south_m"] = enc(
        (request.country_south_m or 0.0) / COORD_SCALE
    )
    data["encrypted_country_east_m"] = enc(
        (request.country_east_m or 0.0) / COORD_SCALE
    )
    data["encrypted_country_west_m"] = enc(
        (request.country_west_m or 0.0) / COORD_SCALE
    )

    return repository.create_company(
        data=data
    )
```

The administrator can then assign an employee to the company and run a location check for that employee to determine whether he is:

- at home
- at the office
- in the country

The check can, however, be run only after the employee has shared her/his encrypted location with the service provider.

5.5.3. Employees server computations

The employee must complete a form with the following information:

- home coordinates: latitude, longitude, and radius;
- current address: latitude and longitude.

After this, when the user submits the form, the data is encrypted and transmitted to the service provider. As this is a demo, the current location is manually inserted by the user, but in a real application, it should be obtained automatically when the employee opens the PC. Also, the current location should be sent again to the service provider immediately after the company requests a compliance check, such that the current location is always up to date.

```
class SaveLatestLocationRequest(
    BaseModel
):
    company_id: int
    current_lat_m: float
    current_lon_m: float
    home_lat_m: float
    home_lon_m: float
    home_radius_m: float = 500.0
```

5.5.4. Encrypted Compliance Check

The first intuition would have been to use the Euclidean distance to find if the user is inside the radius:

$$(\text{lat} - \text{office_lat})^2 + (\text{lon} - \text{office_lon})^2 \leq r^2 \tag{5.15}$$

However, as can be observed, it involves two multiplications, which are expensive, and a comparison, which is not even supported.

Therefore, a trick was used. The client coordinates are checked directly to determine whether they fit inside the bounding box:

$$\text{home_lat} - r \leq \text{current_lat} \leq \text{home_lat} + r \tag{5.16}$$
$$\text{home_lon} - r \leq \text{current_lon} \leq \text{home_lon} + r \tag{5.17}$$

Rewritten to be more appropriate for the encrypted scheme, the conditions are as follows:

$$\text{margin_north} = (\text{home_lat} + r) - \text{current_lat} \geq 0 \tag{5.18}$$
$$\text{margin_south} = \text{current_lat} - (\text{home_lat} - r) \geq 0 \tag{5.19}$$
$$\text{margin_east} = (\text{home_lon} + r) - \text{current_lon} \geq 0 \tag{5.20}$$
$$\text{margin_west} = \text{current_lon} - (\text{home_lon} - r) \geq 0 \tag{5.21}$$

Therefore, the margin values tell us whether the user is at home or not. To be at home, all margins must be positive. An important note is that the comparison with zero is not performed by the service provider. Instead, the service provider sends the encrypted

margins to the company server, which will further handle the decision regarding the status.

```
cc = load_crypto_context(
    metadata["crypto_context"]
)

load_eval_mult_keys(
    cc,
    metadata["eval_mult_keys"]
)

enc_home_north_margin = cc.EvalSub(
    load_ciphertext(
        payload["encrypted_home_north"]
    ),
    load_ciphertext(
        payload["encrypted_current_lat"]
    ),
)

enc_home_south_margin = cc.EvalSub(
    load_ciphertext(
        payload["encrypted_current_lat"]
    ),
    load_ciphertext(
        payload["encrypted_home_south"]
    ),
)

enc_country_east_margin = cc.EvalSub(
    load_ciphertext(
        payload["encrypted_country_east"]
    ),
    load_ciphertext(
        payload["encrypted_current_lon"]
    ),
)

enc_country_west_margin = cc.EvalSub(
    load_ciphertext(
        payload["encrypted_current_lon"]
    ),
    load_ciphertext(
        payload["encrypted_country_west"]
    ),
)
```

As can be noted, this implementation involves only subtractions, which are negated

additions and are cheap, with no multiplications and no comparisons. An identical approach was used to check whether the employee is at the office and in the country. Because the code is repetetive for the three checked regions, only a representative sample was provided in the above code.

5.5.5. Threshold Decryption and Fusion

As there are three bounding boxes (office, country, home), each with four limits (south, north, east, west), there is a total of 12 encrypted margins.

The encrypted margins are transmitted from the service provider back to the clients. Usually, in a threshold scheme, one of the parties is called the 'lead", as it is always the one who initiates the process (encrypts first and decrypts first), and the other is called the 'main", which finishes and orchestrates the process. In our scenario, the client is the lead, and the company is the main.

```
up = openfhe_multiparty_runtime.user_partial_decrypt_scalar
cp = openfhe_multiparty_runtime.company_partial_decrypt_scalar
```

Therefore, the 12 encrypted margins are first sent to the client for decryption, which obtains partial results that are then forwarded to the company:

```
up_home_north = up(
    encrypted_margins["encrypted_home_north_margin"]
)
up_home_south = up(
    encrypted_margins["encrypted_home_south_margin"]
)
up_home_east = up(
    encrypted_margins["encrypted_home_east_margin"]
)
up_home_west = up(
    encrypted_margins["encrypted_home_west_margin"]
)
```

The company receives the partial results from the employee, as well as the encrypted results from the service provider. The company decrypts the margins in a similar manner:

```
cp_home_north = cp(
    encrypted_margins["encrypted_home_north_margin"]
)
cp_home_south = cp(
    encrypted_margins["encrypted_home_south_margin"]
)
cp_home_east = cp(
    encrypted_margins["encrypted_home_east_margin"]
)
cp_home_west = cp(
    encrypted_margins["encrypted_home_west_margin"]
)
```

The partial results are then fused by the company server:

```
home_north_m = fuse(
    up_home_north,
    cp_home_north
)
home_south_m = fuse(
    up_home_south,
    cp_home_south
)
home_east_m = fuse(
    up_home_east,
    cp_home_east
)
home_west_m = fuse(
    up_home_west,
    cp_home_west
)
```

5.5.6. Decision Logic

For the employee to be inside a zone, all margins must be greater than zero. If the minimum of the margins is greater than zero, then all of them are greater than zero:

```
home_margin = min(
    home_north_m,
    home_south_m,
    home_east_m,
    home_west_m
)
```

If the margin is zero, it means that the employee is on the boundary. However, it must be taken into consideration that CKKS introduces a small error. Therefore, the following tolerance is used:

```
EPSILON = 1e-3
```

Instead of comparing the minimum boundary value with zero, it is compared with the tolerance:

```
value >= -EPSILON
```

The added tolerance also helps in keeping the results correct for edge cases, such as when the employee sits in the corner of the office or home. In this design, the tolerance comes in favour of the employee. And, as it will be discussed in the testing and validation chapter, all edge cases were manually tried and the results were in accordance with the plaintext values, meaning that the threshold was not too large.

5.6. Detection of Account Takeover

5.6.1. Pipeline

For the development of the thesis, the classical machine learning pipeline was followed, including dataset analysis, handling the missing values, feature engineering, training and testing. The only particularity comes at the training and testing section which is more special as Homomorphic Encryption was introduced.

However, the model is relatively simple, as no complex models could have been tried as the inference would have not been possible on HE. Moreover, because of the limited memory available, only a model was chosen for the final pipeline.

Another particularity is that because of the highly imbalanced dataset, much care was necessary when splitting it. As it can be observed, after choosing the hyperparameters, the model was trained again on both the training and validation set, before testing on the test set. While this is not the best practice, it was necessary as the positive samples were very limited.

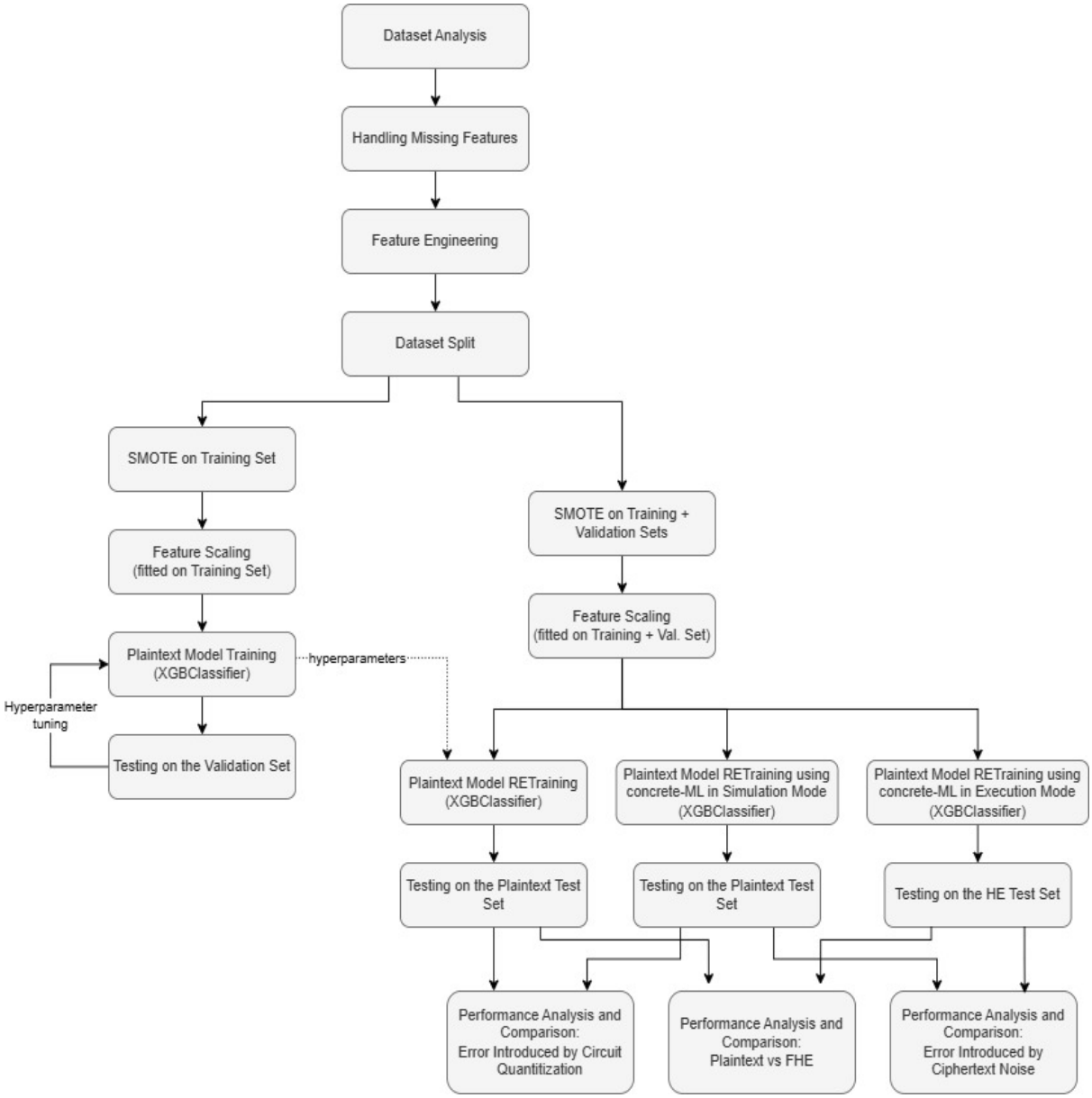


Figure 5.4: Pipeline for the detection of account takeover

5.6.2. Dataset Analysis & Memory Usage Optimization

The RBA dataset is a public dataset that contains 31,269,264 login attempts from 4,304,857 unique users, collected from a single sign-on (SSO) service in Norway. The data was collected between February 2020 to February 2021.

The dataset has the following format, except that two features were not uploaded, namely:

- User Agent String
- Round-Trip Time

For 96% of the samples, **RTT** was missing, and **User Agent String** did not provide much more information, as the same information already appeared in the features **Browser Name and Version**, **OS Name and Version**, and **Device Type**.

It was chosen to discard these features because the notebook runs inside Docker,

which has a maximum capacity of 24 GB. Because of this limitation, when the memory reached its maximum capacity, the kernel stopped. Therefore, the features discussed above were the most suitable to be discarded.

```
LOAD_COLS = [
    "Login Timestamp",
    "User ID",
    "Country",
    "Region",
    "City",
    "Browser Name and Version",
    "OS Name and Version",
    "Device Type",
    "ASN",
    "IP Address",
    "Is Attack IP",
    "Login Successful",
    "Is Account Takeover",
]
```

A few samples from the uploaded dataset are as follows:

	Login Timestamp	User ID	IP Address	Country	Region	City	ASN	Browser Name and Version	OS Name and Version	Device Type	Login Successful	Is Attack IP	Is Account Takeover
0	2020-02-03 12:43:30.772	-4324475583306591935	10.0.65.171	NO	-	-	29695	Firefox 20.0.0.1618	iOS 13.4	mobile	False	False	False
1	2020-02-03 12:43:43.549	-4324475583306591935	194.87.207.6	AU	-	-	60117	Chrome Mobile 46.0.2490	Android 4.1	mobile	False	False	False
2	2020-02-03 12:43:55.873	-3284137479262433373	81.167.144.58	NO	Vestland	Urangsvag	29695	Android 2.3.3.2672	iOS 7.1	mobile	True	False	False
3	2020-02-03 12:43:56.180	-4324475583306591935	170.39.78.152	US	-	-	393398	Chrome Mobile WebView 85.0.4183	Android 4.1	mobile	False	False	False
4	2020-02-03 12:43:59.396	-4618854071942621186	10.0.0.47	US	Virginia	Ashburn	398986	Chrome Mobile WebView 85.0.4183	Android 2.2	mobile	False	True	False

Figure 5.5: RBA uploaded dataset

The features of the dataset can be split into four categories:

- network identity
- device fingerprint
- temporal context
- authentication outcome

To further reduce the memory usage, the high-cardinality features were converted to the `category` data type. The `category` data type stores each unique value or string only once, using a lookup table, and each row points to the corresponding entry in the lookup table. This way, full Python objects do not need to be created for the same string multiple times.

```
df_raw = pd.read_csv(
    DATASET_CSV,
    nrows=NROWS,
    low_memory=False,
    usecols=LOAD_COLS
)

for col in [
    "Country",
    "Region",
    "City",
    "Browser Name and Version",
    "OS Name and Version",
    "Device Type",
    "IP Address"
]:
    df_raw[col] = df_raw[col].astype(
        "category"
    )
```

The memory occupied by the dataset before the conversion to the `category` data type was 17,640 MB, while after the conversion, it was only 4,013 MB. Therefore, the memory usage was reduced by approximately 4.4 times.

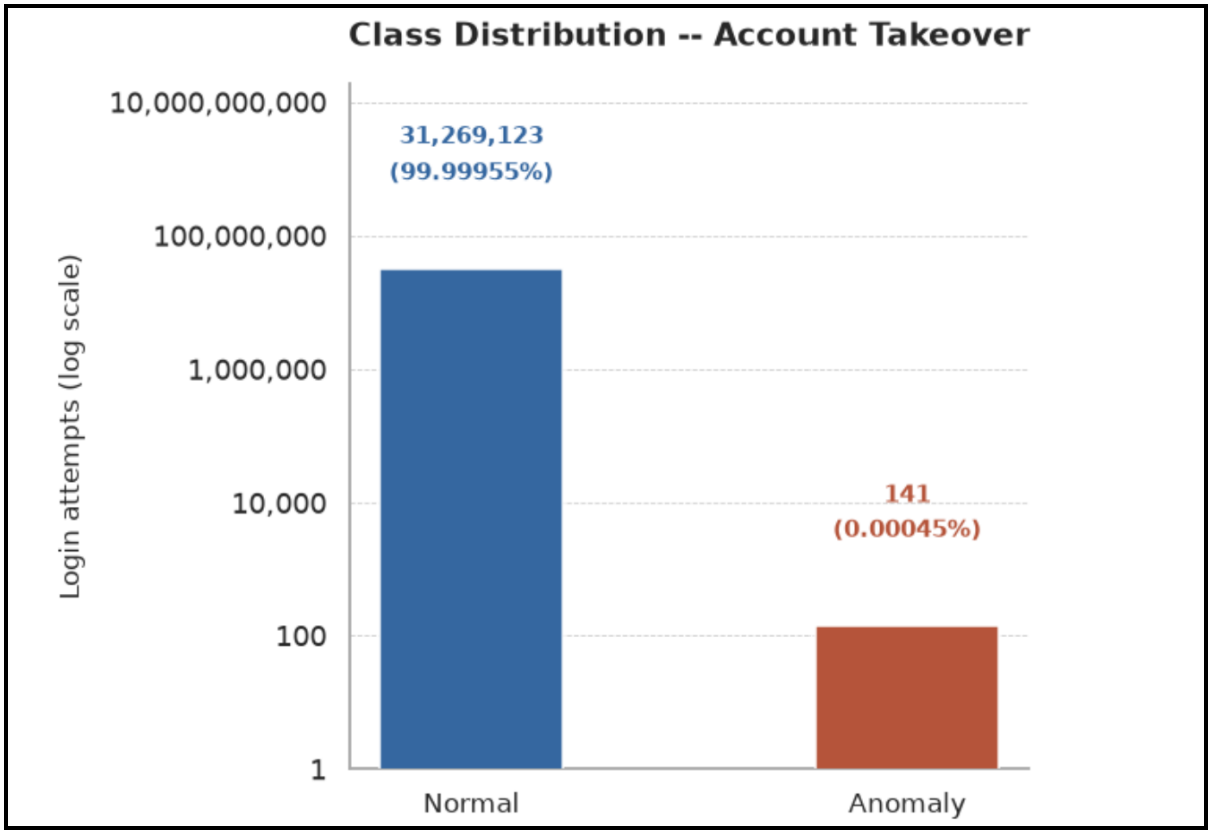


Figure 5.6: Class distribution

A characteristic of the dataset that made the development of this module difficult was its high imbalance. Only 141 records were classified as anomalies, while 31,269,123 were classified as normal. In other words, more than 99.99% of the records represented normal logins.

Before using the data, it was crucial to identify which other features had missing values. The results were as follows:

- Region: had 0.15% missing values
- City: had 0.03% missing values

A temporal analysis was conducted to see, for each month, how many normal and anomalous records were registered, with the initial intention of identifying whether there were periods when attacks peaked. Instead, the results indicated that there were periods when normal traffic peaked and anomalous traffic was completely missing.

Therefore, a trick that was used was to get rid of all data recorded after November 2020. This contributed to solving the two major problems of the dataset: the high imbalance, as only normal traffic was discarded, and the limited memory.

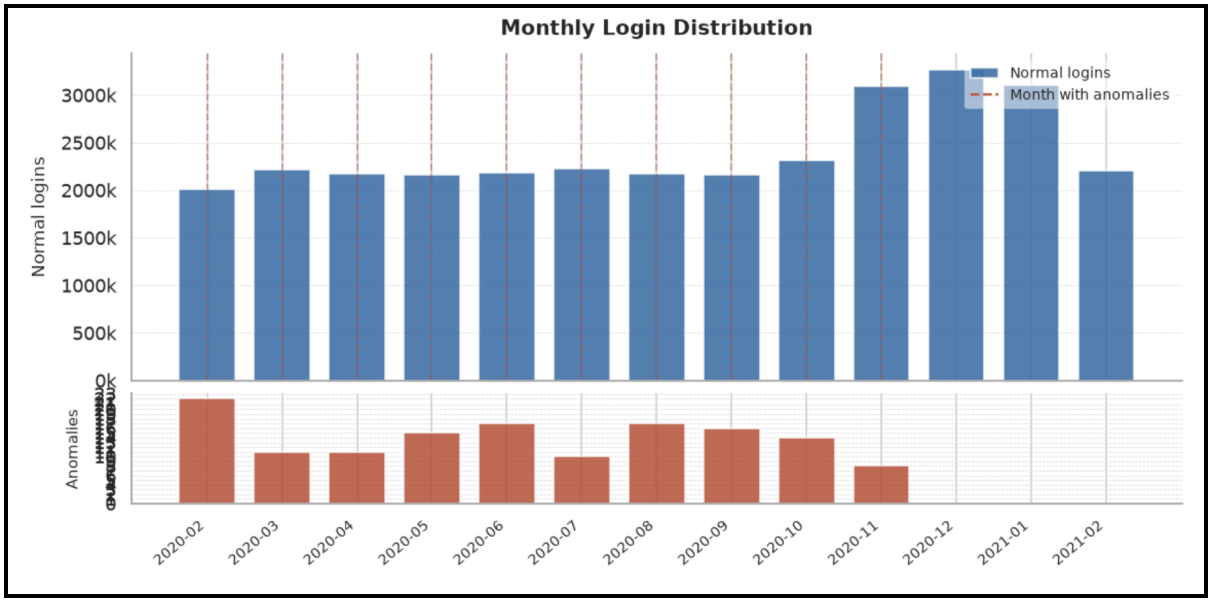


Figure 5.7: Temporal analysis login records

```
_ts = pd.to_datetime(
    df_raw["Login Timestamp"],
    errors="coerce"
)

df_raw = df_raw[_ts < "2020-12-01"].reset_index(
    drop=True
)

del _ts
gc.collect()
```

An analysis was also conducted based on geographic and device characteristics. As can be observed in the results, most of the records are from Norway.

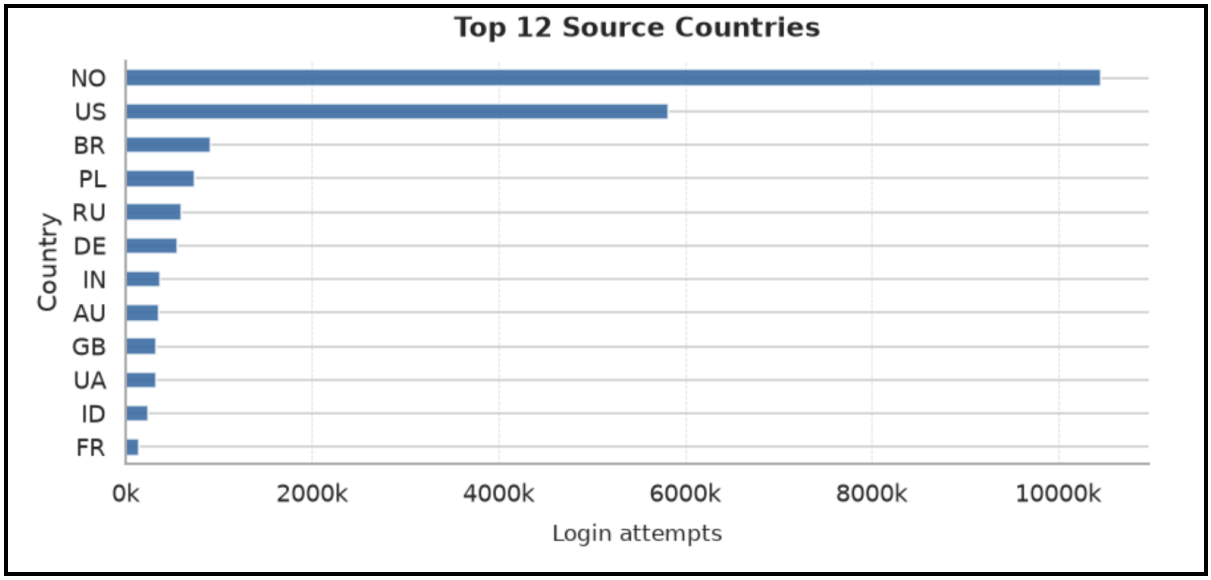


Figure 5.8: Geographic distribution

‘ Additionally, most users connected only once, which is not very helpful for the model, as at least two consecutive logins are required to detect suspicious activity:

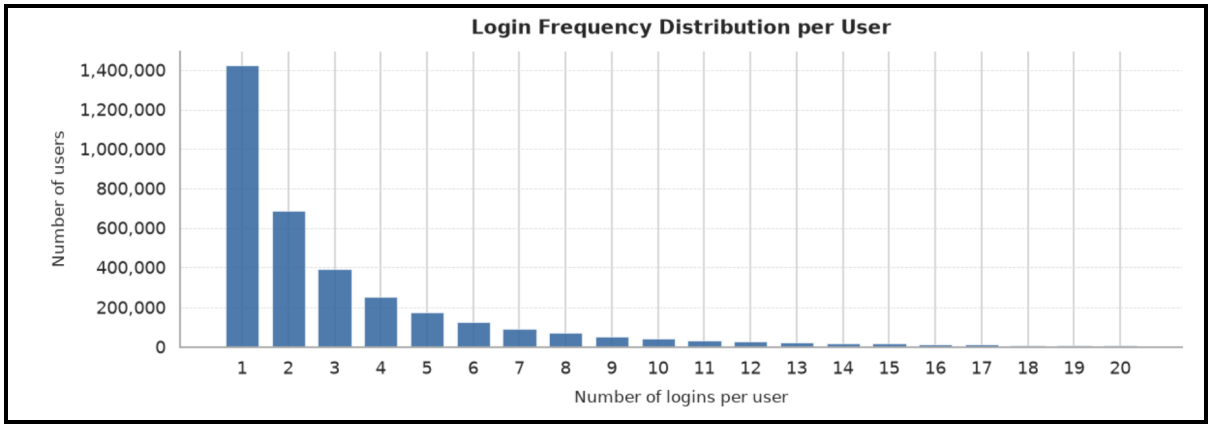


Figure 5.9: Distribution of the number of logins per user

5.6.3. Feature Engineering

As mentioned before, one isolated login carries no temporal context, and at least two consecutive records are needed to detect suspicious changes. Therefore, each record in the dataset was paired with its previous record. The dataset was initially sorted globally by timestamp, but not necessarily by user. To facilitate the pairing, the dataset was sorted first by user and then by timestamp. For each sample, the following features were added by comparing the current login with the previous one:

Feature	Description
distance_km	The approximative distance between the current and previous login locations
speed_kmh	The approximative speed between the two login locations
time_delta_h	The time difference between the current and previous login attempts
same_country	Indicates whether the logins were made from the same country
same_city	Indicates whether logins were made from the same city
same_browser	Indicates whether the same browser was used for both logins
same_os	Indicates whether the same operating system was used for both logins
same_device	Indicates whether the same device type was used for both logins
same_region	Indicates whether the logins were made from the same region
same_asn	Indicates whether the same Autonomous System Number was associated with both logins
prev_login_failed	Indicates whether the previous login attempt was unsuccessful
same_ip	Indicates whether the same IP address was used for both logins
same_ip_prefix	Indicates whether the same IP address prefix was used

Table 5.3: New Features

As it can be noted, the location of the logins is not provided in coordinates. In order to compute the distance between logins, centroids for each present country were defined. The distance between two logins was defined as the distance between the centroids of their coresponding countries.

A problem is that over 1,400,000 logins were isolated, meaning that there were 1,400,000 users who logged in only once. A solution would be to drop all those records, as they negatively affect the model. However, after conducting experiments, it was noted that a substantial number of the records detected as anomalies were dropped, leaving only around 80 anomalous records in the dataset. As a consequence, this idea was abandoned.

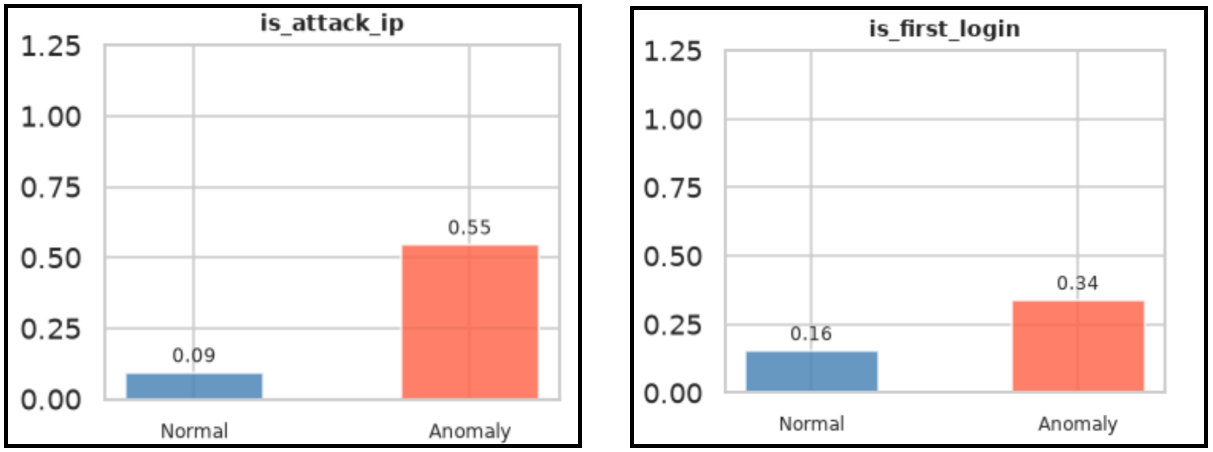
A better alternative to the previous solution was to fill the delta features of the first login with neutral values that indicate identical feature values, for example, the same operating system and a distance of 0. Evidently, this can wrongly influence the model, but at least none of the anomaly samples are discarded.

Other features added were:

Feature	Description
`hour_of_day`	Indicates the hour of the day of the current login
`is_weekend`	Indicates whether the current login occurred during the weekend
`is_first_login`	Indicates whether the current login is the first login of the user

Table 5.4: Additional Features

For each feature, the means of the normal and anomalous samples were compared to determine which features were more representative. Among the results, the most relevant features, for which the discrepancies were high, were the following:



(a) `is_attack_ip` mean

(b) `is_first_login` mean

Figure 5.10: Distribution of the `is_attack_ip` and `is_first_login` features

For the `Is Attack IP` feature, the mean among anomalous samples was 0.55, while for normal samples it was only 0.09.

Additionally, for the `is_first_login` feature, the mean among anomalous samples was 0.34, while for normal samples it was only 0.16. This indicates that anomalous logins are more frequently isolated.

5.6.4. Train-Test Split

The dataset was sorted again by timestamp and split as follows:

- 80% was used for the development set:
  - 80% of the development set was used for the training set
  - 20% of the development set was used for the validation set
- 20% was used for the test set

Therefore, talking about the entire dataset, 64% was used for training, 16% for validation, and 20% for testing.

This splitting prevents data leakage between records. For example, if the dataset were shuffled and then split, a current record might use a login that occurred in the future.

The distribution between the development and test sets is illustrated in the table below:

Metric	Training Set	Test Set
Samples	18,155,979	4,538,995
Anomalies	125	16
Positive rate	0.0007%	0.0004%

Table 5.5: Distribution of samples

The model is trained on the training set, and the validation set is used to choose the best hyperparameters. K-fold cross-validation was not possible because SMOTE needed to be applied to the training set, which generates synthetic samples from the existing ones. This could have caused data leakage, and the validation score would have been almost perfect. To address this problem, the hold-out method was implemented and combined with grid search. To make it more practical, the model was trained on a subsample of the training set and evaluated on the validation set. After hyperparameter tuning, the model was trained again on the full training set.

After this, the model was tested again on the validation set to obtain a baseline. In the end, the model was ultimately trained on the full development set (training + validation) to prepare it for testing.

This workflow is meant only for the plaintext variant of the model. After identifying the best hyperparameters, the corresponding model from Concrete-ML is trained on its development set using those hyperparameters, or at least aims to do so. Currently, poorer hyperparameters are used to make it run faster.

5.6.5. Handling Class Imbalance

As discussed earlier, the dataset is extremly unbalanced, with more than 99.99

Therefore, to handle this imbalance, multiple approaches were considered, including SMOTE (synthetic minority oversampling), undersampling of the majority class, and using the `scale_pos_weight` parameter from XGBoost.

Undersampling was discarded because deleting negative samples could lead to losing valuable information, which could reduce the model's performance.

The chosen method was to use the `SMOTE` (Synthetic Minority Oversampling Technique) is applied. SMOTE uses existing samples from which interpolates their features to obtains new synthetic samples. It must be noted that is very important to apply it only on the training set, otherwise it will cause data leakage.

5.6.6. Model Training

The chosen model is the XGBClassifier, as discussed in Chapter 4. It was trained twice. The first time, it was trained in the normal manner on plaintext data.

```
scaler = StandardScaler()

X_train = scaler.fit_transform(
    X_train_raw
)
X_test = scaler.transform(
    X_test_raw
)

del X_train_raw, X_test_raw
gc.collect()

PT_PARAMS = dict(
    n_estimators=300,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=1,
    gamma=0.0,
    reg_lambda=1.0,
    scale_pos_weight=pos_weight,
    eval_metric="aucpr",
    random_state=SEED,
    n_jobs=-1,
    verbosity=0,
)

pt_xgb = xgb.XGBClassifier(
    **PT_PARAMS
)
pt_xgb.fit(
    X_train,
    y_train
)
```

The standard scaler standardizes the features. More precisely, it computes the mean and variance for each feature using the training samples and then applies the transformation to the samples from both the training, validation and test sets:

$$x' = \frac{x - \mu}{\sigma} \tag{5.22}$$

This scaling ensures that the features have a comparable scale and that differences in scale do not negatively influence the prediction. For this model, scaling is not necessarily required for the plaintext model, but it is required for the homomorphically encrypted circuits.

The `n_estimators` parameter defines the number of boosting trees that the model has. Each tree is built to correct the mistakes of the previous one.

The `max_depth` parameter sets the maximum depth of all branches of a tree. A high depth can lead to better predictions, but at the same time, it can lead to overfitting. However, when choosing this parameter, it was necessary to take into account that inference must also run on the encrypted version, and a high depth means multiple sequential operations, which increase the noise of the ciphertext.

The `learning_rate` parameter controls how much each tree contributes to the final result.

The `subsample` parameter controls what percentage of the training set the trees can use. Each tree randomly and independently selects 80% of the given training set.

The `colsample_bytree` parameter controls what percentage of the features each tree has access to. Similarly to the previous parameter, it randomly selects the features, and each tree makes this selection independently.

The `min_child_weight` parameter determines whether a new leaf can be created based on the minimum sum of instance weights required in that leaf. A higher value makes the model more conservative, while setting it to 1 allows leaves to be created as long as their total instance weight is at least 1.

The `gamma` parameter specifies how much the loss must be reduced to allow a new leaf to be created.

The `reg_lambda` parameter controls L2 regularization.

The `scale_pos_weight` parameter is a factor multiplied by the gradients of the underrepresented class.

The `eval_metric` parameter defines the metric used to evaluate the model during training.

The `random_state` parameter is used as the seed for selecting the subsets discussed when explaining `subsample` and `colsample\_bytree`.

Setting `n_jobs` to -1 means that all available CPU cores are used to construct the trees.

Setting `verbosity` to 0 prevents the model from displaying logging information.

Secondly, the model was trained to be FHE-compatible using the FHE `XGBClassifier` from the Concrete-ML library. It is a version of `XGBClassifier` that is compiled by Concrete-ML into an FHE circuit. Currently, the parameters are set differently from those of the plaintext model to reduce the training and inference times.

```
FHE_PARAMS = dict(
    n_bits=6,
    n_estimators=40,
    max_depth=4,
    learning_rate=0.1,
    min_child_weight=1,
    gamma=0.0,
    scale_pos_weight=fhe_pos_weight,
    random_state=SEED,
    n_jobs=-1,
)

fhe_xgb = FHEXGBClassifier(
    **FHE_PARAMS
)

fhe_xgb.fit(
    X_fhe,
    y_fhe
)

rng_c = np.random.default_rng(
    SEED
)

cal_idx = rng_c.choice(
    len(X_fhe),
    min(1_000, len(X_fhe)),
    replace=False
)

X_calib = X_fhe[cal_idx]
```

5.6.7. Inference

The 20% of the dataset that was held out is used for testing in three different manners:

- on plaintext
- in simulated FHE mode
- in real FHE mode

Firstly, the model is tested on plaintext data so that a true evaluation of the model itself is obtained, more precisely, an inference-time and performance baseline.

Secondly, in FHE simulation mode, the model is tested on the FHE circuit, but the data is not encrypted. It uses the same arithmetic and quantization as real FHE, so that only the error introduced by quantization can be measured.

Lastly, in real FHE mode, the dataset is encrypted and evaluated on the FHE circuit. Any gap in performance between FHE simulation and real FHE would indicate

discrepancies introduced during encrypted execution, while the difference between real FHE mode and plaintext mode is the final and most important result of this module, as it indicates whether FHE deserves attention and should be used in real deployments.

5.7. Transmission Protocol

5.7.1. Interface Definition

The file `service_provider.proto` represents the gRPC contract that must be respected and shared by both the client and the service provider. The content of the file is as follows:

```
syntax = "proto3";

package thesis;

service ServiceProviderGateway {
    rpc Call(ServiceProviderRequest)
        returns (ServiceProviderResponse);

    rpc UploadConcreteKeys(stream ConcreteKeyChunk)
        returns (ConcreteKeyUploadResponse);
}

message ServiceProviderRequest {
    string method = 1;
    string path = 2;
    string json_payload = 3;
}

message ServiceProviderResponse {
    int32 status_code = 1;
    string json_payload = 2;
}

message ConcreteKeyChunk {
    bytes data = 1;
}

message ConcreteKeyUploadResponse {
    int32 status_code = 1;
    string json_payload = 2;
}
```

In `ServiceProviderGateway`, there are two RPCs defined, namely `Call` and `UploadConcreteKeys`.

The `Call` RPC receives as input a message containing a method, a path, and a JSON payload, and returns a status code and a JSON payload. This is a unary RPC,

meaning that one request from the client triggers one response from the server. This RPC is defined generically to allow different types of operations to be sent.

The UploadConcreteKeys RPC is a streaming method that was defined for sending the evaluation keys, which are extremely large. The client sends multiple chunks of data, while the server sends one response.

When the images are built, the Dockerfiles of both the client and the service provider run grpc_tools.protoc in order to generate:

- service_provider_pb2.py, which contains the message classes
- service_provider_pb2_grpc.py, which contains the stub and servicer base classes

The two files are placed in the app/generated/ directory.

5.7.2. Service Provider support for gRPC

The service provider contains all implementations for supporting gRPC in the file service_provider_app/app/grpc_server.py. The class ServiceProviderGatewayServicer implements both methods, Call and UploadConcreteKeys.

The method Call acts as a router. It receives the message, extracts the fields, and redirects the payload to the appropriate service.

A sample from the method is as follows:

```
if method == "POST" and clean_path == "/service/poi/nearest-k":
    dto = ServiceProviderNearestKRequest(
        **payload
    )
    result = poi_service.nearest_k_by_category(
        dto
    )
    return _json_response(
        200,
        result
    )
```

The method UploadConcreteKeys iterates asynchronously through the chunks of data that it receives and appends them to the previously received parts of the keys. When all chunks are received, it calls the method execute_concrete_upload_keys.

5.7.3. Client support for gRPC

The client communicates with the service provider through the file client_app/app/infrastructure/grpc_client.py. The method call_service_provider(method,path,payload) opens a grpc.aio.insecure_channel to the configured host and port, sends the request to the service provider, and waits for a response. The time limit for receiving a response was set to 600 seconds, taking into account that HE schemes are very slow.

The method upload_concrete_keys_to_service_provider iterates through the key bytes, splits them into 4 MB chunks, and uploads one chunk during each iteration.

Chapter 6. Testing and Validation

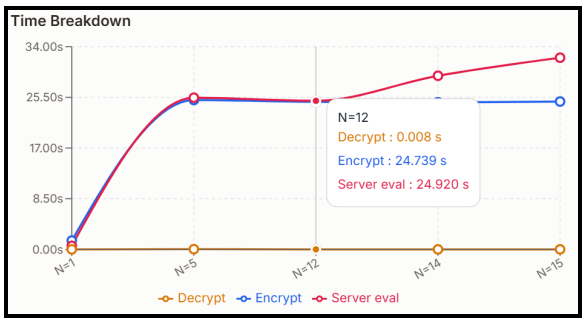
6.1. Finding the Nearest Point of Interest

To measure the performance of this module, a simulation was conducted. More specifically, after the user entered their coordinates, all categories were tested to measure metrics such as encryption time, decryption time, server evaluation time and the communication overhead introduced by the exchange of data between the parties.

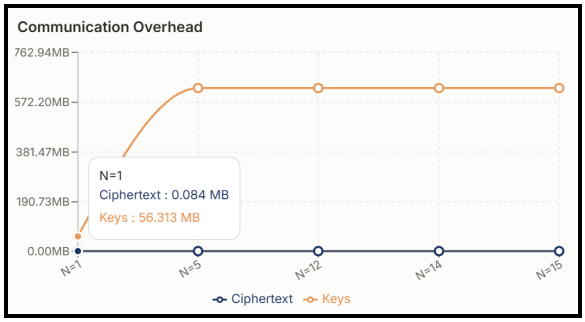
HE Performance Metrics							
N	**Scheme**	**Encrypt (s)**	**Server eval. (s)**	**Decrypt (s)**	**Total (s)**	**Ciphertext (MB)**	**Eval. keys (MB)**
1	Concrete	1.478	0.597	0.003	3.490	0.084	56.313
	OpenFHE SS	32.239	0.403	0.450	40.172	14.008	83.788
5	Concrete	25.049	25.435	0.041	68.985	0.063	626.283
	OpenFHE SS	23.196	0.348	29.163	57.925	14.008	68.784
12	Concrete	24.739	24.920	0.008	64.440	0.063	626.283
	OpenFHE SS	16.872	0.355	38.968	61.837	14.008	68.784
14	Concrete	24.668	29.117	0.008	67.980	0.063	626.283
	OpenFHE SS	17.018	0.381	37.240	59.389	14.008	68.784
15	Concrete	24.797	32.145	0.010	71.472	0.063	626.283
	OpenFHE SS	17.065	0.363	37.360	59.537	14.008	68.784

Table 6.1: HE performance metrics

As the numbers in the table do not tell much in this form, they were further displayed as graphs. The graph on the left represents a breakdown of the time spent by the TFHE scheme from Concrete on the main processes: encryption, decryption, and server evaluation. On the other hand, the graph on the right focuses on memory usage, namely the overhead introduced by the exchange of ciphertexts from the server to the client and the exchange of keys from the client to the server.



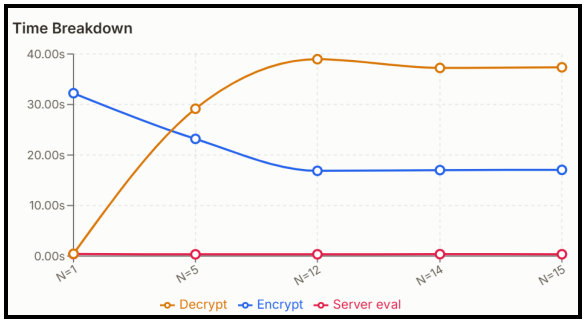
(a) Concrete latency



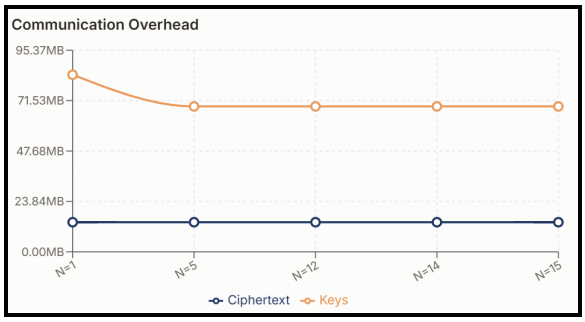
(b) Concrete overhead

Figure 6.1: Concrete performance results

The results indicate that, as the number of POIs, (N), increases, the encryption and decryption times remain relatively constant, while the server evaluation time increases sublinearly. Regarding the memory overhead, it seems to remain constant.



(a) OpenFHE latency



(b) OpenFHE overhead

Figure 6.2: OpenFHE performance results

For the combined CKKS+FHEW schemes from OpenFHE, all metrics remain constant, except for decryption. This is because the selection of the argmin could not be moved to the server side, as the server would need access to a specific evaluation key for the FHEW scheme. While that key could have been sent from the client to the server, it would have broken the security guarantee. If FHEW had been used from the beginning, that key would not have been necessary for performing the scheme-switching operation.

Interestingly, it seems that the encryption time is the highest for N=1, but this is the result of the first time that the client generates the keys and crypto context. For the other values of N, the crypto context was cached, so the latency does not include the setup overhead. Moreover, it can be observed that the decryption curve grows somehow logarithmically. The important note is that the size of the vector on which the encrypted user coordinates are replicated has a size k, which increases with N. It must be larger than or equal to N, but k has the restriction of being a power of 2. Therefore, for N=12, N=14, and N=15, the value of k is 16 for each. This means that they perform the same amount of work. This is the reason for which the plot seems to be constant at these values.

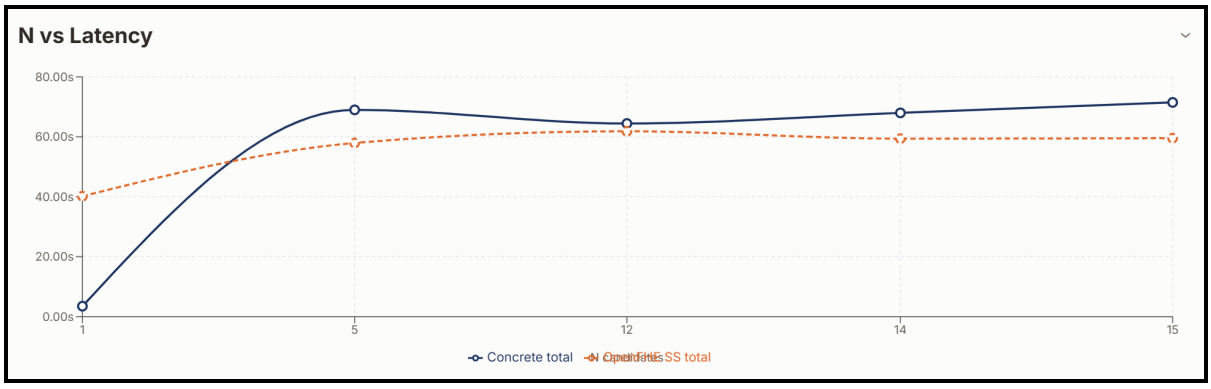


Figure 6.3: Latency

Not to forget the scope of this module, which was to compare the performance of the two libraries, both were plotted on the same graph. As can be seen, the OpenFHE library performs much better than the Concrete library.

However, the average deviation in distance when recommending the closest POI is much smaller for Concrete (0.08 km) than for OpenFHE (0.58 km).

Result Accuracy								
N	Plaintext		Concrete			OpenFHE SS		
	POI returned	Distance to user (km)	POI returned	Distance to user (km)	Deviation from plaintext (km)	POI returned	Distance to user (km)	Deviation from plaintext (km)
1	Pediatrie 2	0.9185	Pediatrie 2	0.9185	0.0000	Pediatrie 2	0.9185	0.0000
5	Liceul Teologic Unitarian János Zsigmond	0.2838	Liceul Teologic Unitarian János Zsigmond	0.2838	0.0000	Colegiul Economic Iulian Pop	0.3997	0.1160
12	deScris penshop	0.1609	Colop	0.4662	0.3053	Stampile la minut	0.4600	0.2991
14	Farmec	0.3145	Parfumeria Dialissa	0.3416	0.0271	Gerovital	2.8179	2.5035
15	Muzeul Catolic	0.0722	Unic Art Gallery	0.1782	0.1060	Muzeul Catolic	0.0722	0.0000
Average	—		Concrete		0.0877	OpenFHE SS		0.5837

Table 6.2: Accuracy results

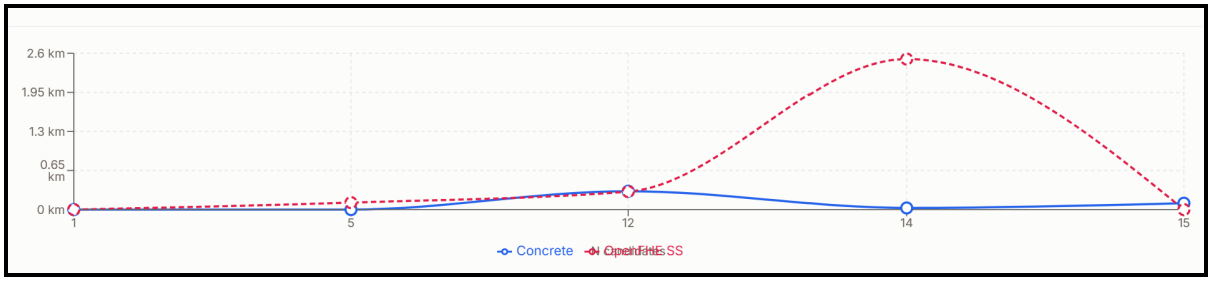


Figure 6.4: Accuracy of the results

6.2. Location Compliance Verification: Theoretical Protocol

To ensure that the pipeline where the compliance check is run using HE works correctly, it was compared against a plaintext verification. To ensure that the system works correctly in all edge cases, more experiments were run, as follows:

- the user was inside the company office area;
- the user was at home, inside the country;
- the user was outside the country;
- the user was at the office, on the margin of the radius declared in the company policy.

In the UI, the results of both the plaintext and HE verifications can be seen:

Status	FHE Result	Plaintext Result	Correctness
At office	No	No	Yes
At home	Yes	Yes	Yes
In country	Yes	Yes	Yes

Table 6.3: Compliance verification results

An important note is that, because of time constraints, automated tests were not implemented for this module. However, the test cases were manually verified through the UI.

6.3. Detection of Account Takeover

While the final result, in terms of recall and ROC-AUC, might be of greater importance or interest, I believe that the intermediate discoveries deserve an equal amount of attention, as they might be helpful in finding new ways to improve the model.

The correlation matrix for the model is shown in the table above:

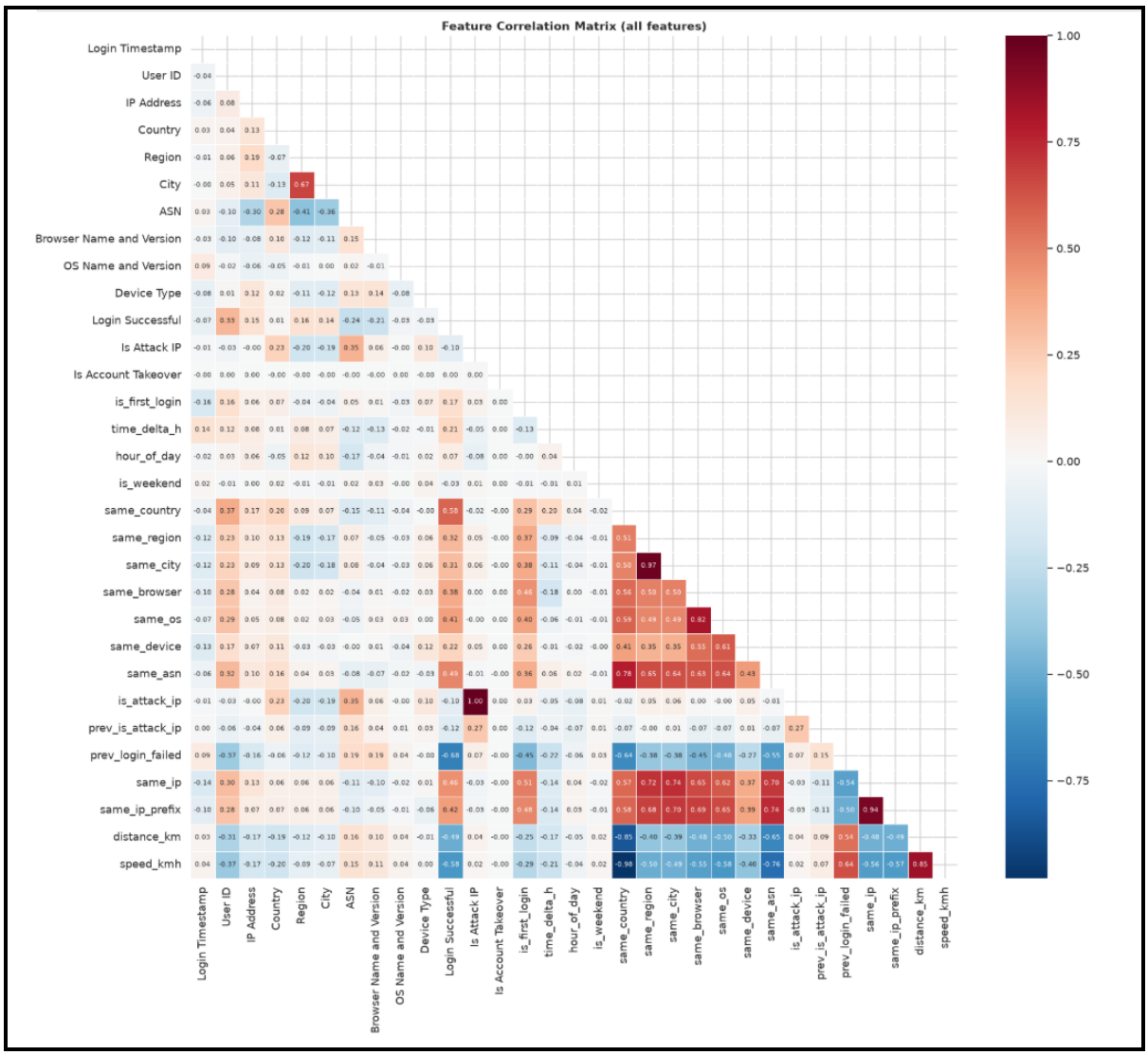


Figure 6.5: Feature correlation matrix

The correlation matrix indicates that the engineered features are highly correlated with each other, while the initial login features are almost independent.

Grid search was used to find the best hyperparameters for the model by training on the training set and evaluating on the validation set. The best hyperparameters found were:

```
{'n_estimators': 200, 'max_depth': 4, 'learning_rate': 0.05}
```

The results obtained on the validation set were:

Metric	Value
Recall (anomaly)	0.5600
ROC-AUC	0.840
False positives	208,930
False negatives	11

Table 6.4: Performance on the Validation Set

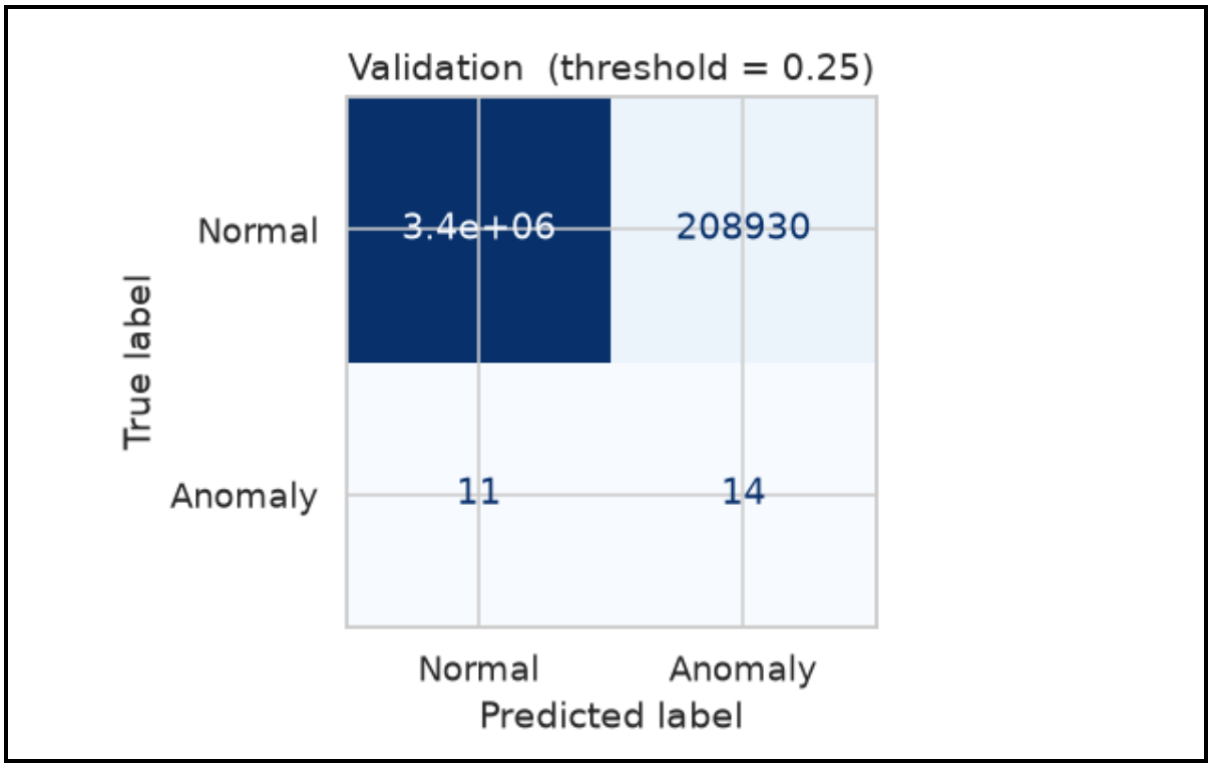


Figure 6.6: Confusion matrix for Validation Set

The ROC curve illustrates the trade-off between increasing the true positive rate and increasing the false positive rate. The best threshold for the model was found to be 0.25.

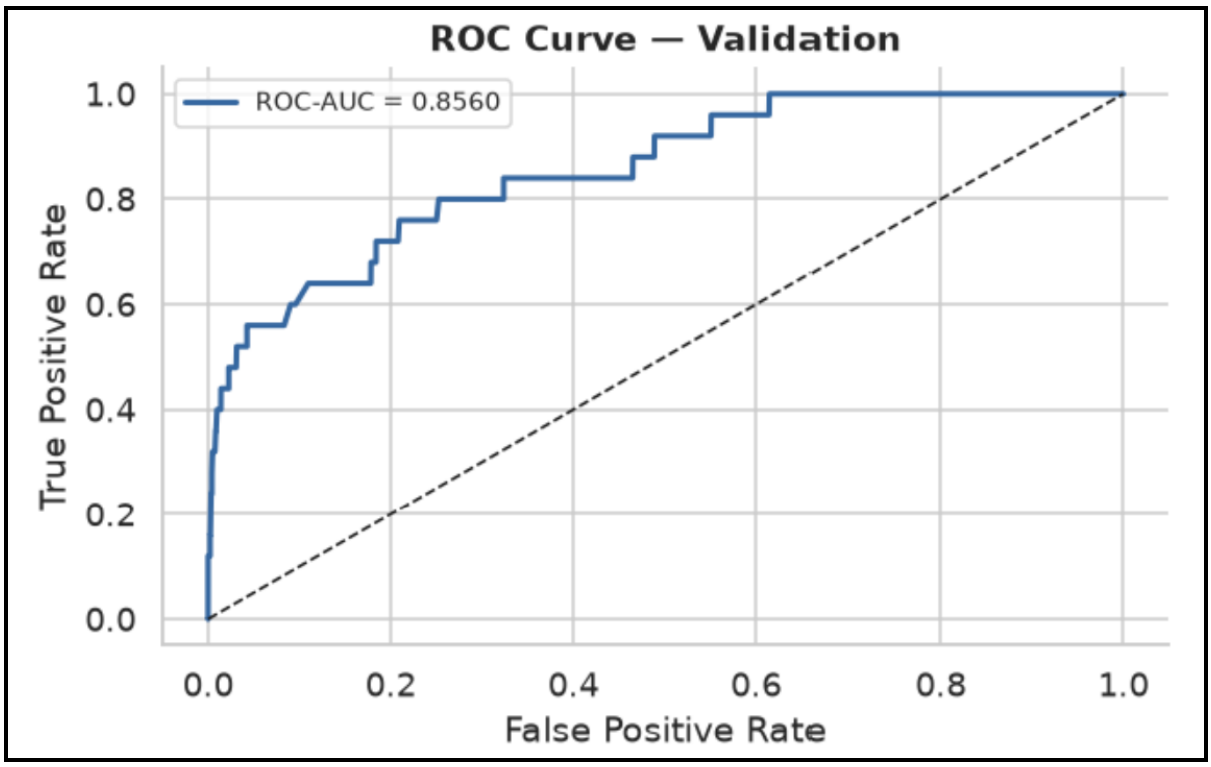


Figure 6.7: ROC Curve

As expected, not all features have the same importance in the model, and they were ranked as follows:

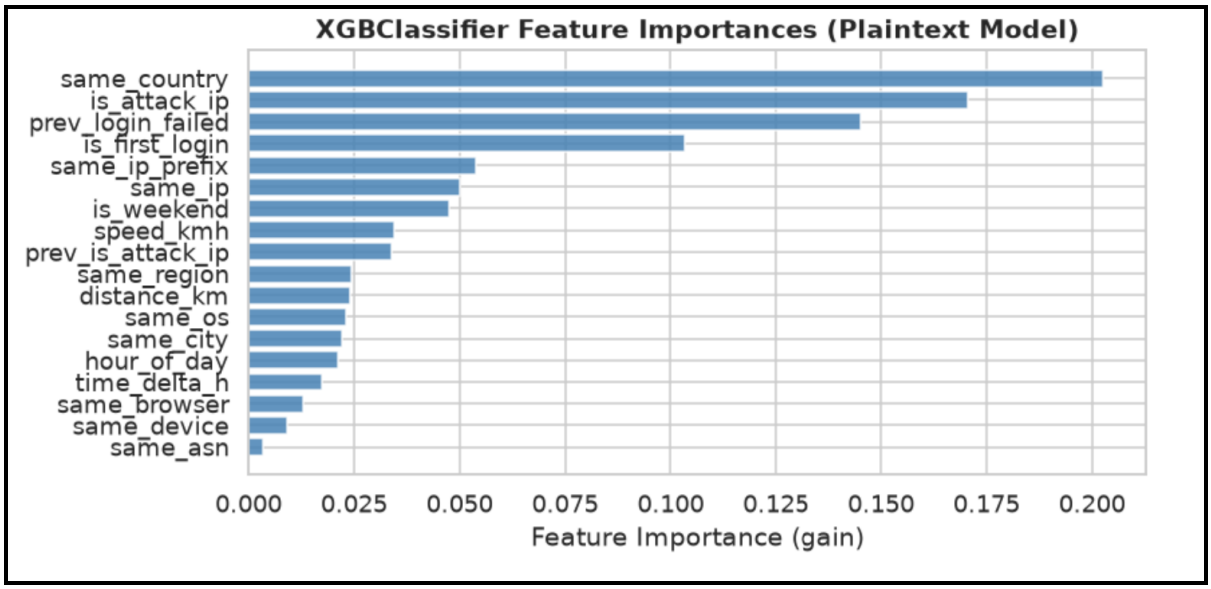


Figure 6.8: Feature importance ranking

The above figure indicates that the most important features were `same_country`, `is_attack_ip`, `prev_login_failed`, and `is_first_login`. At the opposite end, `same_asn` and `same_device` were the least predictive.

Finally, the results obtained by the plaintext model on the test set were:

Plaintext XGBoost Test Results	
Metric	**Value**
Recall (anomaly)	0.6875
Accuracy	0.9598
ROC-AUC	0.9455
Latency	0.30 μs/sample

Table 6.5: Performance of the plaintext XGBoost model

The results on the test set usually indicate good generalization of the model. However, not to be very optimistic, I would personally attribute this outcome to the fact that the validation set represented only 20% of the development set, which was 80% of the original set, meaning that the validation set was 16% of the original, while the test set was 20% of the original. This is important because the anomaly samples are very rare, and one bad classification can have a big impact on the final results, especially for smaller datasets.

The confusion matrix obtained on the test set is shown below:

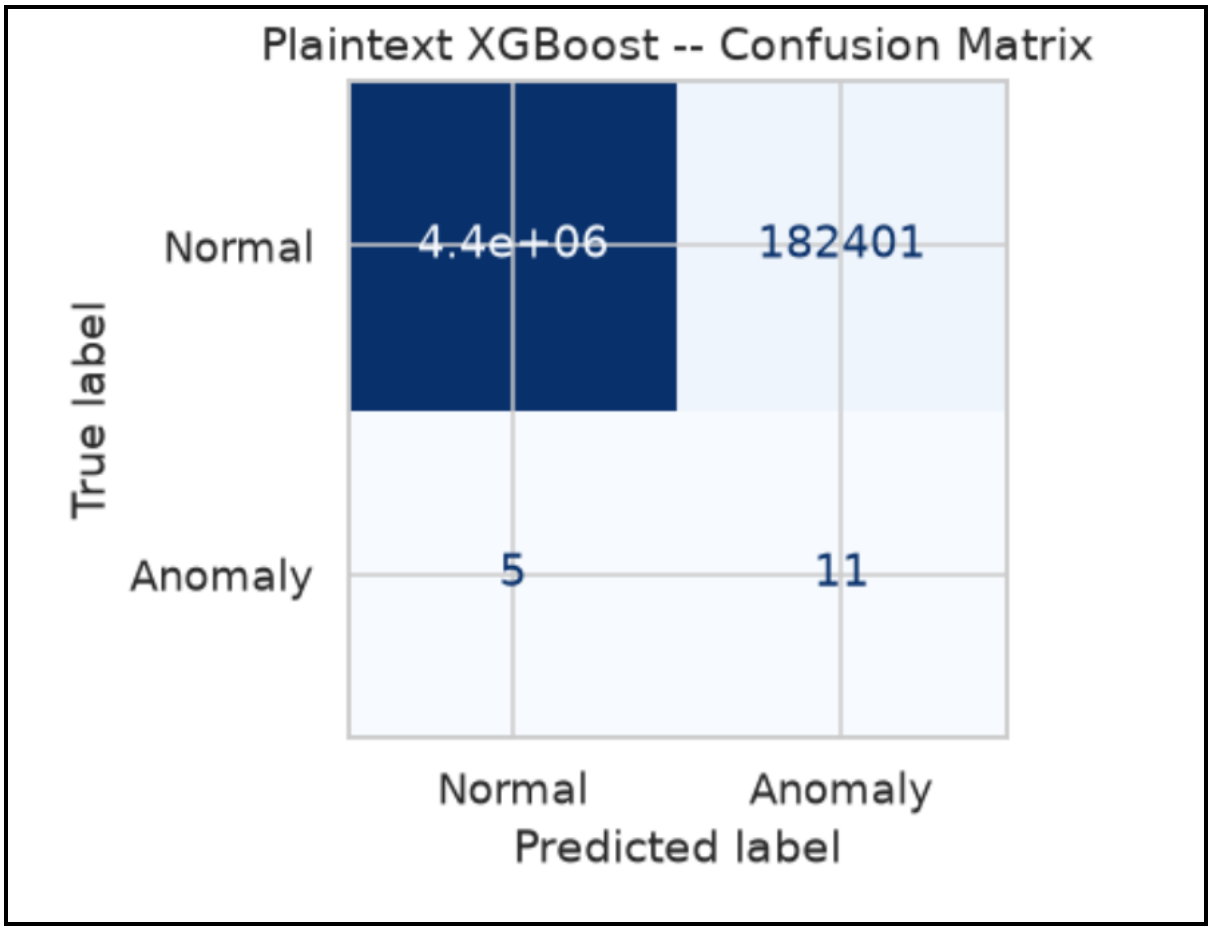


Figure 6.9: Confusion matrix for Test Set

As this model was not necessarily the main scope of this thesis, we will further analyse the results from the fully homomorphic encryption version.

For the FHE version, two testing scenarios were considered. The first one is called FHE simulation, and the inference was performed using Concrete-ML on plaintext data

to observe the error introduced by quantization for the TFHE circuit. The second one is real FHE, where the inference is performed on encrypted data. By comparing plaintext inference with FHE simulation, one can observe the quantization error, while by comparing FHE simulation with real FHE, one can observe the error introduced by ciphertext noise. Finally, and most importantly, by comparing plaintext inference with real FHE, one can observe the final performance gap and the inference time per sample.

Chapter 7. User's Manual

7.1. Development Environment

The application runs inside Docker containers. Therefore, the user does not need to manually install Python, Node.js, or any cryptographic libraries. However, the host machine must meet a few requirements.

Firstly, the supported operating systems are Windows 10/11 with WSL2 enabled, macOS 12+, and Linux. The Concrete library is natively compatible with Linux and macOS, but not with Windows. As the PC on which this thesis was developed used Windows, a workaround was required, namely running the application inside Docker with WSL2.

Docker Engine and Docker Compose are the only software components that must be manually installed by the user. Docker Engine must be version 24.0 or later, while Docker Compose must be version 2.x. The machine must have at least 20 GB of RAM and at least 10 GB of free disk space for storing the keys.

The user should also have a web browser in which to open the user interface, such as Google Chrome, Microsoft Edge, or Mozilla Firefox.

The network ports used are the following:

Port	Component	Service Used For
8000	Client Server	HTTP, FastAPI
8001	Service Provider	HTTP, FastAPI
50051	Service Provider	gRPC
5432	Main POI database	`thesis_db`, PostGIS
5433	Authentication database	`thesis_auth_db`
5434	User location database	`thesis_users_db`
5435	Company and compliance database	`thesis_company_db`
5173	Frontend	Vite

Table 7.1: Network ports

7.2. Libraries and Dependencies

Both the client and the service provider run on Python 3.12. Some of the installed libraries are the following:

- `tenseal`, which provides homomorphic encryption schemes (e.g. CKKS)
- `concrete-python`, which provides the TFHE scheme
- `openfhe`, which provides the CKKS and FHEW schemes
- `torch`, a dependency required by `concrete-python`

- `fastapi`, a Python framework used to build the REST API
- `pydantic`, which is used for data validation
- `psycopg[binary]`, a PostgreSQL driver
- `PyJWT`, which is used to generate and verify JWT tokens
- `pwdlib[argon2]`, which is used for password hashing
- `scikit-learn`, which is used for classical machine learning
- `numpy`, which is used for numerical computing
- `pandas`, which is used for processing tabular data
- `protobuf`, which is used to define the messages send by gRPC
- `grpcio`, which provides the gRPC runtime
- `grpcio-tools`, which is a library that takes the Protocol Buffers definition and generates Python classes
- `httpx`, which provides an asynchronous HTTP client

The above list does not contain all the installed libraries, but only the most relevant ones.

7.3. Execution Configuration

The file `compose.yml` defines the following services, which are run when the application is started:

- the container `thesis_db`, which uses the image `postgis/postgis:16-3.4`
- the container `thesis_auth_db`, which uses the image `postgres:16`
- the container `thesis_users_db`, which uses the image `postgres:16`
- the container `thesis_company_db`, which uses the image `postgres:16`
- the container `thesis_service_provider`, which is built from `service_provider_app/Dockerfile`
- the container `thesis_client`, which is built from `client_app/Dockerfile`

Another service, `thesis_jupyter`, is also declared, but it must be built and started independently.

7.4. Starting the Application

A sequence of steps must be followed by the user to start the application, as discussed below.

The user must compile the six Concrete FHE circuits before starting the application so that the required artifacts are generated beforehand:

```
docker exec -it thesis_client python /app/tools/compile_distance_circuit.py
```

The output of this command should generate the following artifacts:

```
server_n1.zip
client_specs_n1.bin

server_n5.zip
client_specs_n5.bin

server_n12.zip
client_specs_n12.bin

server_n13.zip
client_specs_n13.bin

server_n14.zip
client_specs_n14.bin

server_n15.zip
client_specs_n15.bin
```

The artifacts must be copied to the corresponding folder:

```
docker cp thesis_client:/app/.artifacts/concrete/. ./client_app/.artifacts/
    concrete/
```

To build the images for the services defined in `compose.yml`, the user must open a terminal in the root directory. The command to run is the following:

```
docker compose build
```

In a different terminal, from the `frontend` folder, the user must start the development server:

```
npm install
npm run dev
```

The user can then navigate to the following web page to interact with the user interface:

`http://localhost:5173`

To stop the application, the user must run:

```
docker compose down
```

To stop the application and delete all stored data, including the database volumes:

```
docker compose down -v
```

To interact with the third module, the account takeover detection module, the user must start it independently:

```
docker build -f notebooks/Dockerfile -t notebooks-jupyter .

docker run -p 8888:8888 -v "${PWD}/notebooks:/notebooks" notebooks-jupyter
```

The user can open in a web browser at:

`http://localhost:8888`

Chapter 8. Conclusions

8.1. Contributions

This study demonstrated how three different scenarios, implemented in many real-world applications and involving the exposure of geolocation data, can be redesigned to protect the user's location. While this work explored only homomorphic encryption as a method for protecting the data and analysed its feasibility and performance, many other approaches also exist, such as differential privacy and private information retrieval.

Consequently, to summarize my contributions:

- implemented a combination of the CKKS+FHEW and TFHE schemes for the POI search module
- compared the response time, result accuracy, and computational overhead of the CKKS+FHEW approach against TFHE for the POI search module
- designed and implemented an encrypted multiparty computation approach for the location-sharing module using threshold CKKS
- evaluated the performance and accuracy of threshold CKKS against plaintext computation
- designed and implemented an account takeover detection approach using the RBA dataset and compared the performance of plaintext inference against TFHE inference using the Concrete-ML library
- identified the scenarios in which HE is suitable and the cases that deserve particular attention when transitioning to other techniques, such as PIR

8.2. Analysis of the Results

8.2.1. Finding the Nearest Point of Interest

This was the only module that had a negative result. It proved that homomorphic encryption is not the correct approach for finding the closest point of interest, let alone for a fully functional application such as Google Maps.

Although extensive preprocessing was applied to the dataset and many limitations were imposed, such as limiting it to the Cluj-Napoca region and using only specific categories for which the number of samples was known, the functionality of the application remained limited. Additionally, the overhead and long latency introduced by HE cannot justify switching to this approach solely to protect privacy, as it would be impractical when an answer is needed very quickly, such as when driving at high speed.

On the other hand, there might be situations in which a user would like to know their distance to a point without exposing their location or the requested category, and they have enough time to wait, for example, less than five minutes.

8.2.2. Location Compliance Verification: Theoretical Protocol

While the logic for implementing this module was complex and involved extensive communication among the three parties, it was the one that achieved the best results.

For all the scenarios that were tested, the results of the plaintext compliance check were the same as those of the HE compliance check, while the time required to obtain the result was not substantially longer.

8.2.3. Detection of Account Takeover

After completing the development of the account takeover detection approach using the Login Data Set for RBA, I agree with the authors of the dataset that ML approaches are suitable for analysing the data and selecting the relevant features. On the other hand, I believe that the model achieves sufficiently good performance using an ML approach, which the authors discarded as being less effective than the traditional Freeman statistical model.

While the Freeman statistical model might have achieved better performance than the XGBClassifier on this dataset, this result should not be generalised to all datasets. This dataset contains a very small number of positive samples, which made detection particularly difficult. However, for more complex types of attacks and more complex datasets, ML has increasingly started to replace traditional approaches. Even Microsoft stated that its account takeover detection system shifted towards ML-based prediction.

Regarding the use of TFHE, the difference in predictive performance between plaintext and encrypted inference is negligible, but the inference time is significantly higher.

8.3. Future Development

As future work, the major change would be to shift the approach used for POI search from homomorphic encryption to private information retrieval. However, to improve the existing approach, for the CKKS+FHEW scheme, I would try to move the argmin selection to the service provider side in order to reduce the transmission overhead. For Concrete, I would allow the server to contribute to writing the specification file, so that there would be no need to send the POIs to the client during the setup required to generate the file from which the circuit is later built.

Regarding a better user experience, the interface should automatically retrieve the user's location. When the user is asked to enter their home address, they should be able to select it from a list, similarly to Google Maps, instead of manually entering the latitude and longitude coordinates.

For the account takeover detection module, I would split the notebook so that the inference samples are sent from the client to the service provider. While the results in terms of recall and precision should remain similar, the overhead introduced by TFHE is expected to be significantly higher than that of plaintext inference.

Bibliography

[1] A. Aloufi, P. Hu, H. Liu, S. S. Chow, and K.-K. R. Choo, "Universal location referencing and homomorphic evaluation of geospatial query," *Computers & Security*, vol. 102, p. 102137, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404820304107

[2] S. Wiefling, P. R. Jørgensen, S. Thunem, and L. Lo Iacono, "Login data set for risk-based authentication," Jun. 2022. [Online]. Available: https://doi.org/10.5281/zenodo.6782156

[3] S. Wiefling, P. R. Jørgensen, S. Thunem, and L. L. Iacono, "Pump up password security! evaluating and enhancing risk-based authentication on a real-world large-scale online service," *ACM Trans. Priv. Secur.*, vol. 26, no. 1, Nov. 2022. [Online]. Available: https://doi.org/10.1145/3546069

[4] I. Matiushin and V. Korkhov, "Mle-rba: A machine learning-empowered risk-based authentication algorithm," in *Computational Science and Its Applications – ICCSA 2025 Workshops: Istanbul, Turkey, June 30 – July 3, 2025, Proceedings, Part IX*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 325–339. [Online]. Available: https://doi.org/10.1007/978-3-031-97648-3_22

[5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," Cryptology ePrint Archive, Paper 2016/421, 2016. [Online]. Available: https://eprint.iacr.org/2016/421

[6] L. Ducas and D. Micciancio, "FHEW: Bootstrapping homomorphic encryption in less than a second," Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: https://eprint.iacr.org/2014/816

[7] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421

Appendix A. Relevant Code sections

Appendix B. Other Relevant Info

Proofs etc. if any. Otherwise remove this chapter